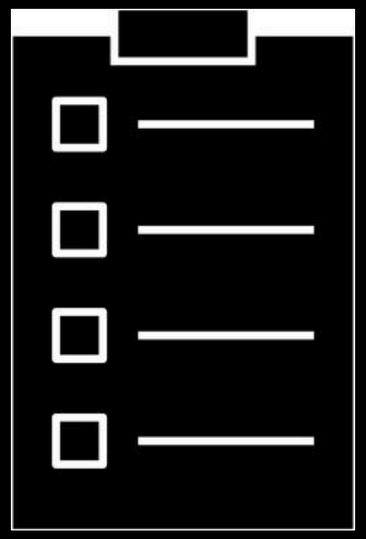
List

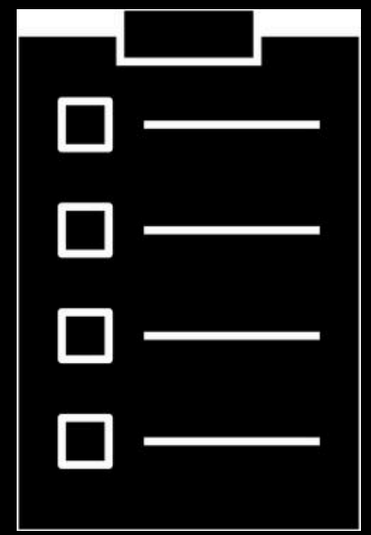


Tiziana Ligorio
Hunter College of The City University of New York

List ADT

What makes a list?

E.g. Play**List**?

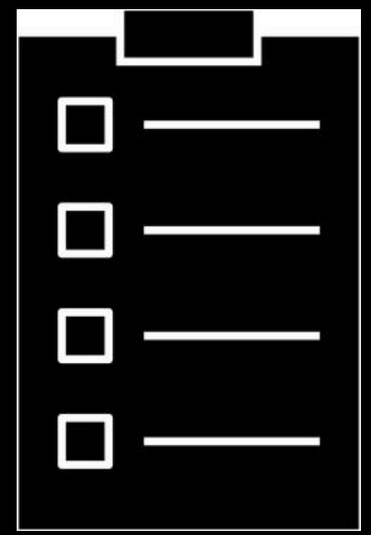


Duplicates allowed or not is not a defining factor

List ADT

What makes a list?

E.g. Play**List**?



Duplicates allowed or not is not a defining factor

ORDER!!!

```cpp
#ifndef LIST_H_
#define LIST_H_

template<class T>
class List
{

public:
    List(); // constructor
    List(const List<T>& a_list); // copy constructor
    ~List(); // destructor
    // assume copy and move operations
    bool isEmpty() const;
    size_t getLength() const;

    //retains list order, position is 0 to n-1, if position > n-1 it inserts at end
    bool insert(size_t position, const T& new_element);
    bool remove(size_t position);//retains list order
    T getItem(size_t position) const;
    void clear();


private:
    //implementation details here


}; // end List

#include "List.cpp"
#endif // LIST_H_
```

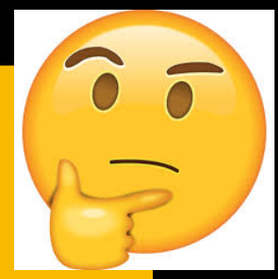
Defines the ADT

Used to represent the size of objects. Can think of it as unsigned it, but it is platform dependent.

Must choose implementation

Implementation

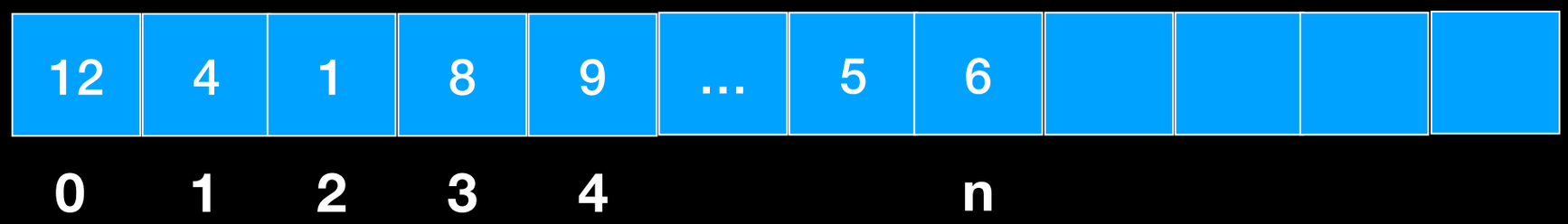
Must preserve order
No swapping tricks



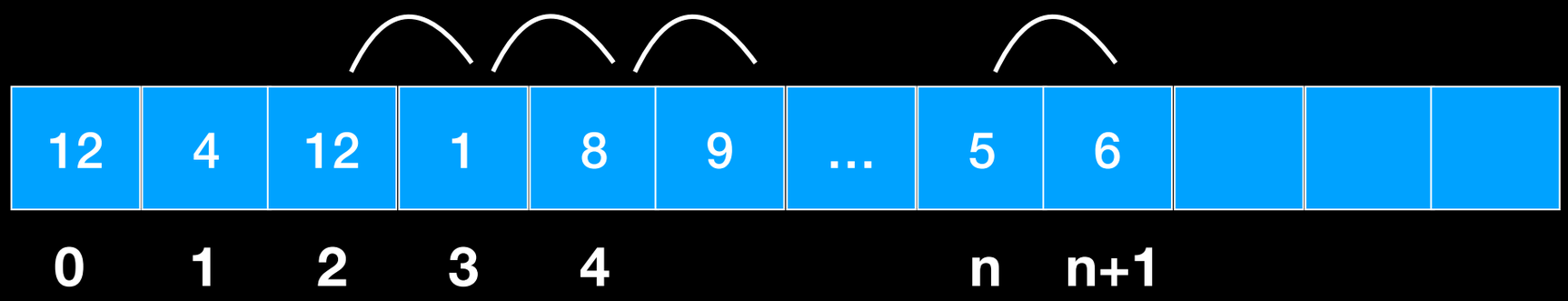
Array?

Linked Chain?

Array

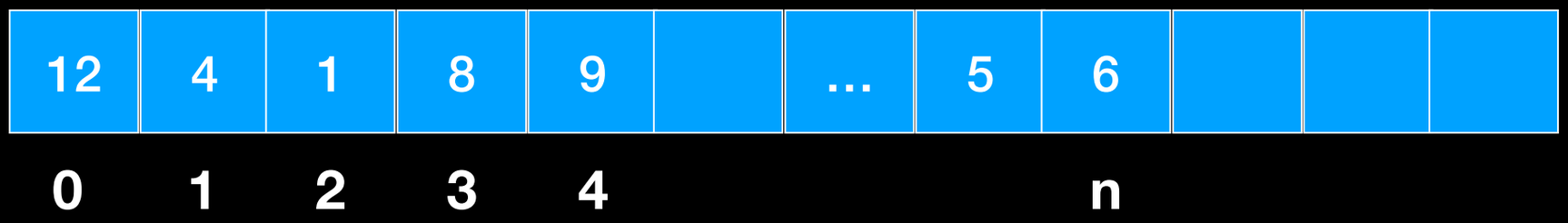



```
insert(2, 12)
```



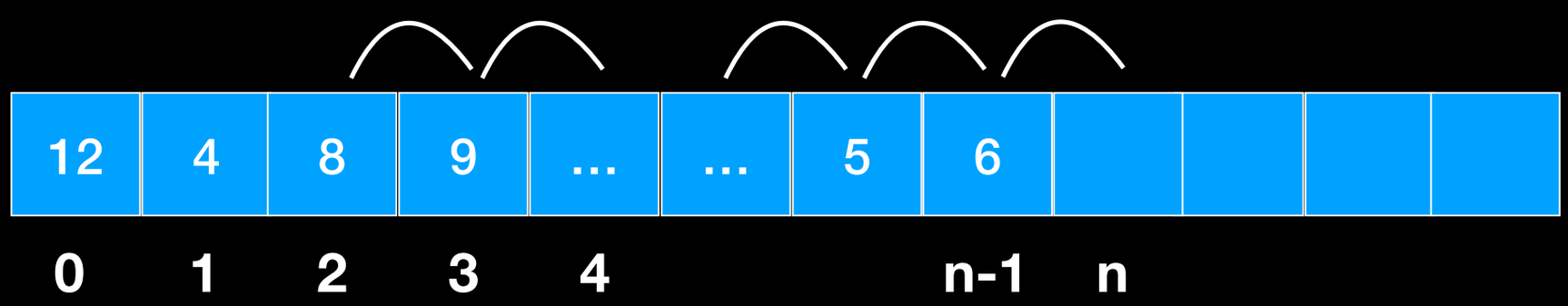
Must shift `n-(position+1)` elements

Array



`remove(2)`

similarly



Must shift `n-position` elements

Array Analysis

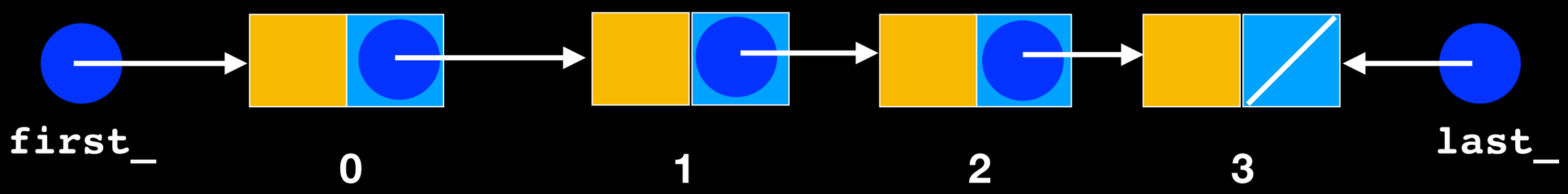
With Array both insert and remove are O(n)

Number of operations depends on size of List

Can we do better?

What makes a list?

Order is implied

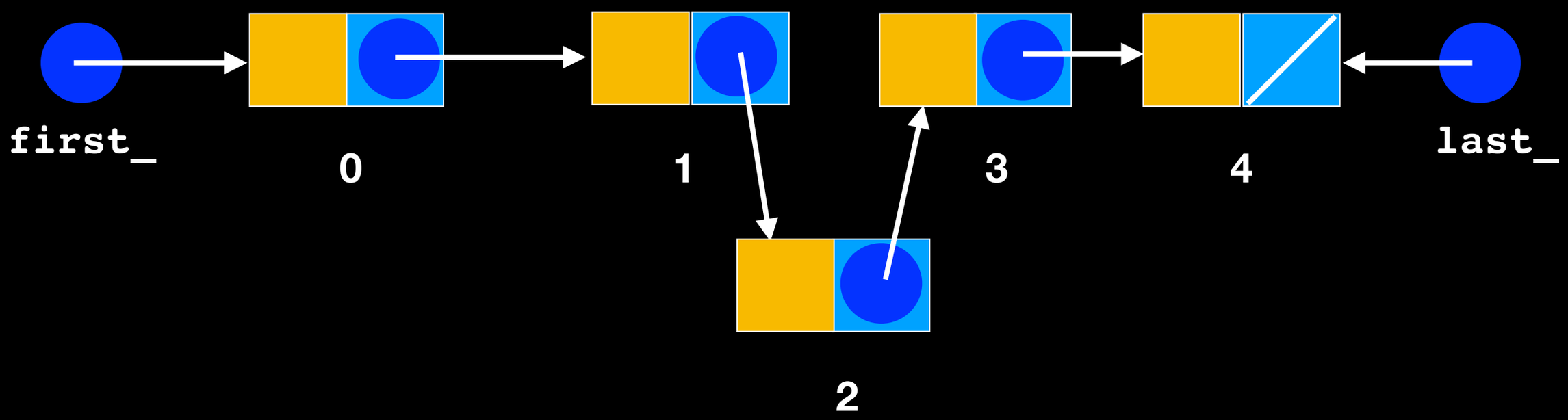


What makes a list?

Order is implied

Insertion and removal from middle retains order

No shifting necessary

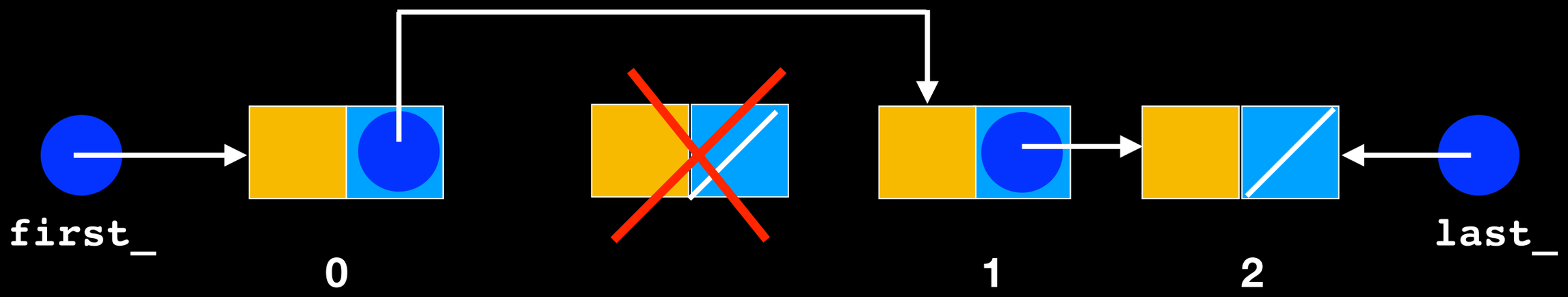


What makes a list?

Order is implied

Insertion and removal from middle retains order

No shifting necessary



What's the catch?

What's the catch?

No random access

As opposed to arrays or vectors with direct indexing

O(n): each insertion and removal must traverse `position+1` nodes

Here too, number of operations depends on size of List

O(1) Constant

O(n) depends on # of items

	Arrays	Linked List
Random/direct access		
Retain order with Insert and remove At the back		
Retain order with insert and remove at front		
Retain order with insert and remove In the middle		

O(1) Constant

O(n) depends on # of items

	Arrays	Linked List
Random/direct access		
Retain order with Insert and remove At the back		
Retain order with insert and remove at front		
Retain order with insert and remove In the middle		

O(1) Constant

O(n) depends on # of items

	Arrays	Linked List
Random/direct access		
Retain order with Insert and remove At the back		
Retain order with insert and remove at front		
Retain order with insert and remove In the middle		

O(1) Constant

O(n) depends on # of items

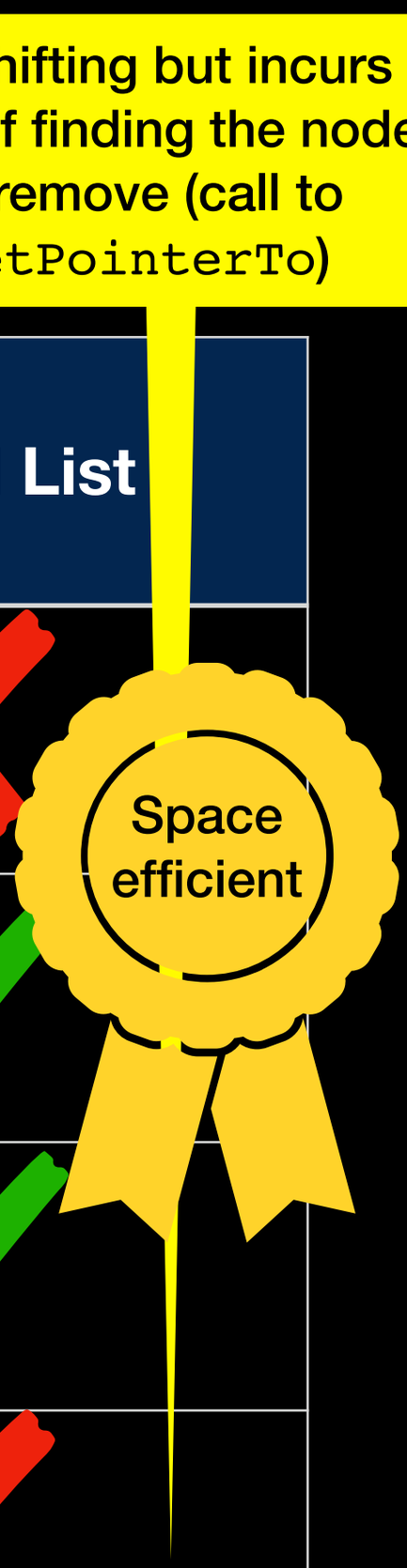
	Arrays	Linked List
Random/direct access		
Retain order with Insert and remove At the back		
Retain order with insert and remove at front		
Retain order with insert and remove In the middle		

O(1) Constant

O(n) depends on # of items

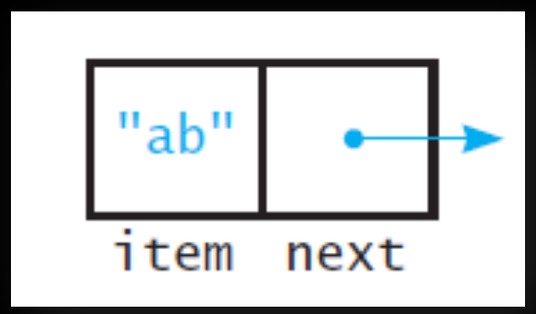
No shifting but incurs cost of finding the node to remove (call to `getPointerTo`)

	Arrays	Linked List
Random/direct access		
Retain order with Insert and remove At the back		
Retain order with insert and remove at front		
Retain order with insert and remove In the middle		



Singly-Linked List

Use the same node as the LinkedBag

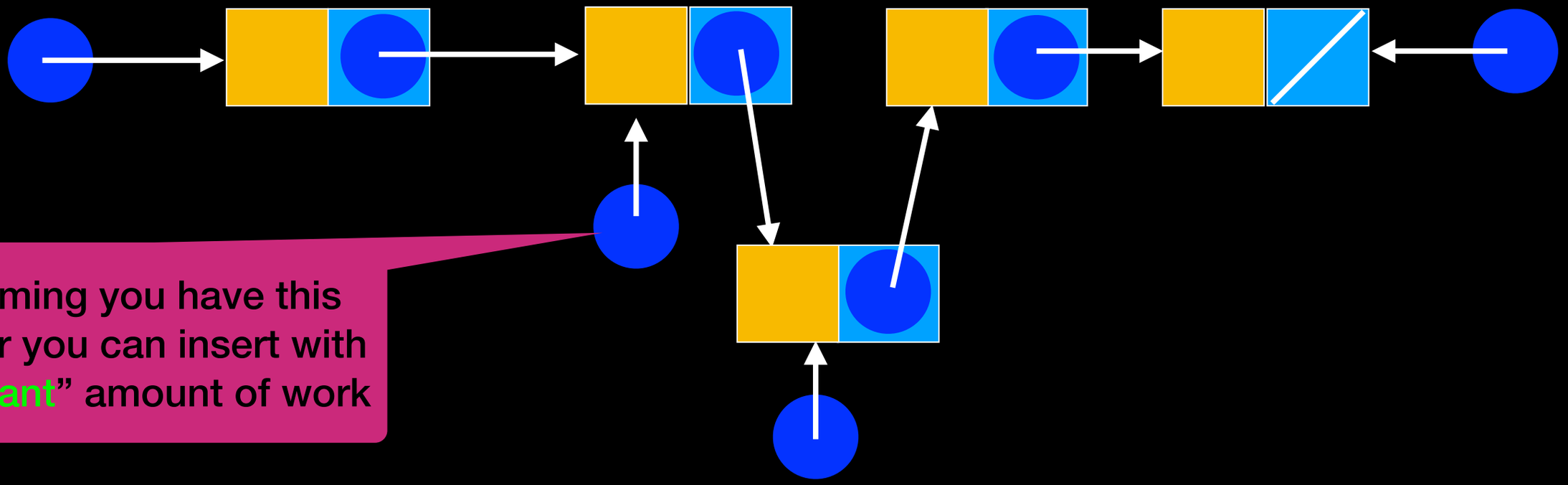


```
#ifndef LIST_H_
#define LIST_H_

#include “Node.hpp"

template<class T>
class List
{

public:
     List(); // constructor
     List(const List<T>& a_list); // copy constructor
     ~List(); // destructor
     bool isEmpty() const;
     size_t getLength() const;

     //retains list order, position is 0 to n-1, if position > n-1 it inserts at end
     bool insert(size_t position, const T& new_element);
     bool remove(size_t position);//retains list order
     T getItem(size_t position) const;
     void clear();


private:
     Node<T>* first_;     // Pointer to first node
     Node<T>* last_;      // Pointer to last node
     size_t item_count_;  // number of items in the list

     Node<T>* getPointerTo(size_t position) const;


}; // end List

#include "List.cpp"
#endif // LIST_H_
```

Additional pointer to last node.

INSERT

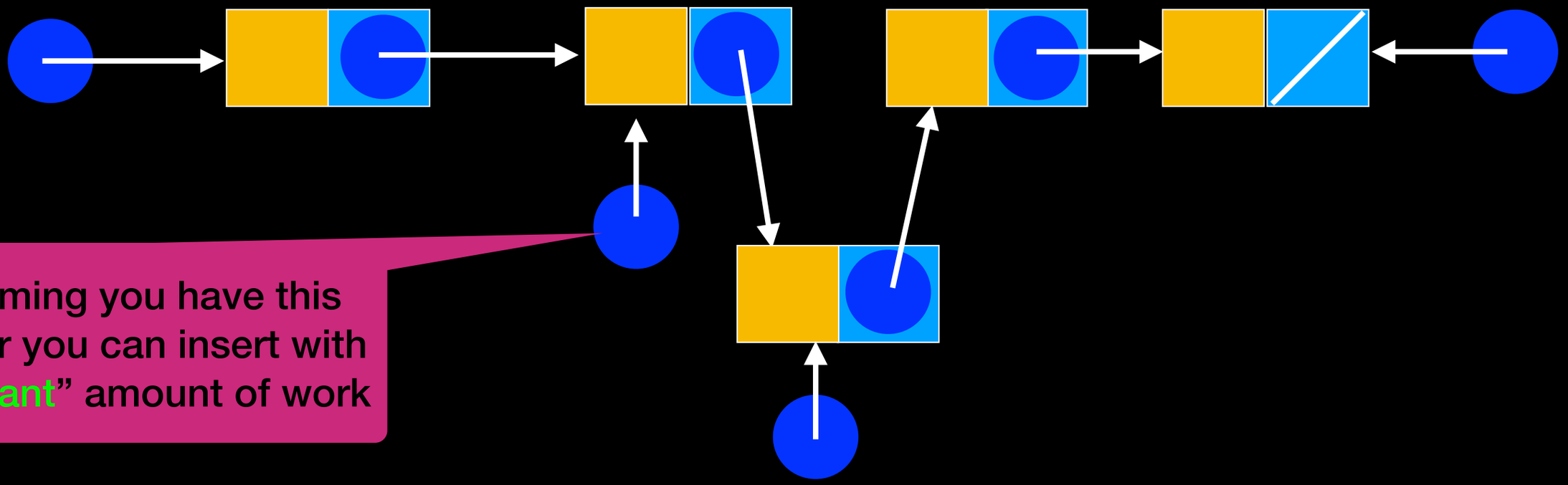
```
void insert(size_t position, T new_element);
```



Assuming you have this pointer you can insert with “constant” amount of work

INSERT

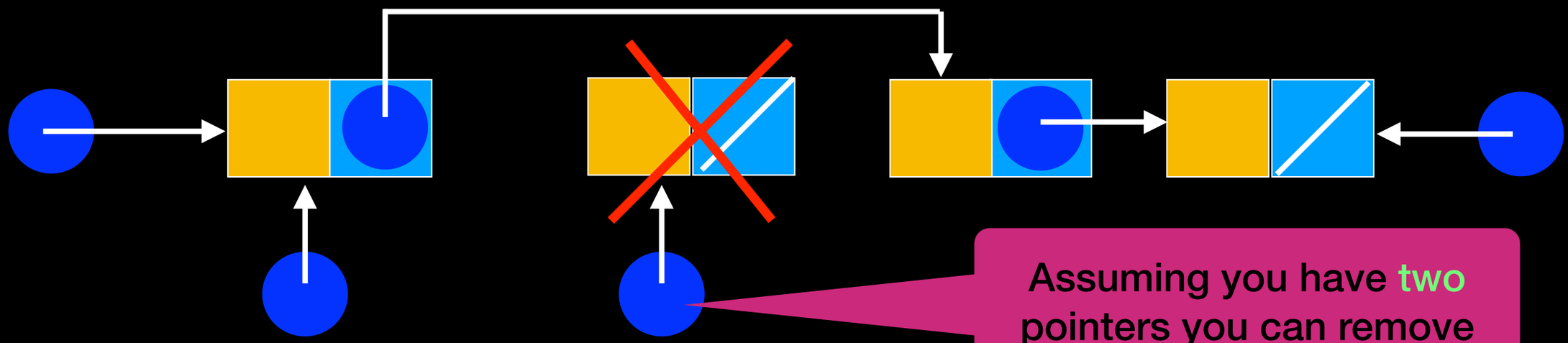
```
void insert(size_t position, T new_element);
```



Assuming you have this pointer you can insert with “constant” amount of work

REMOVE

```
void remove(size_t position);
```



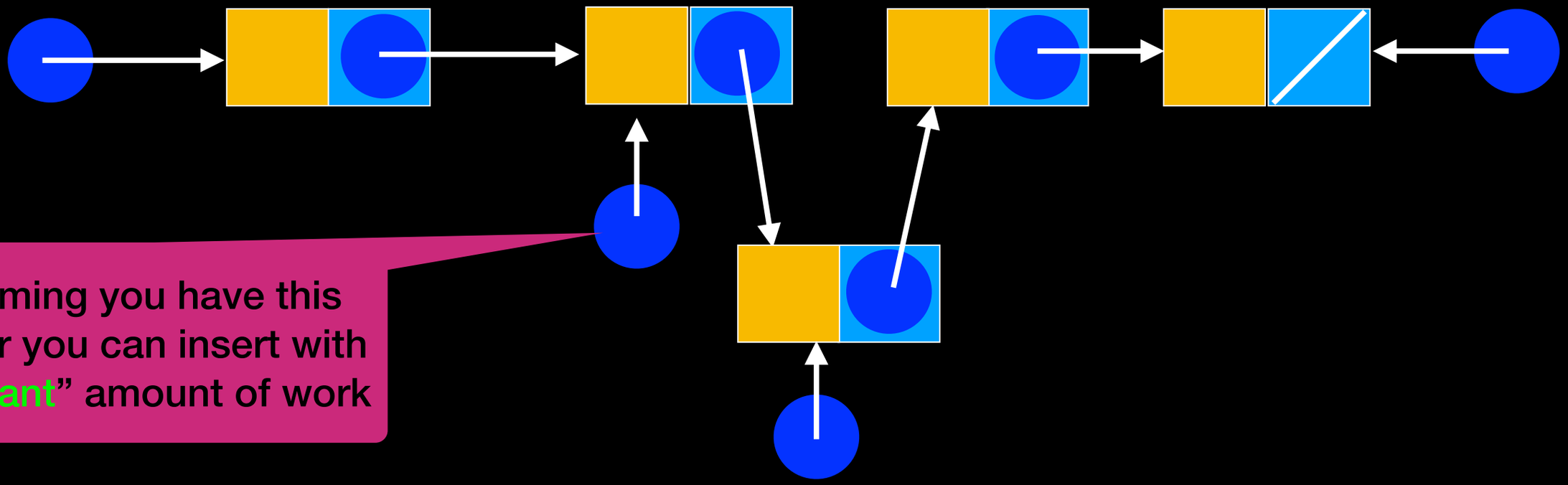
Assuming you have two pointers you can remove with “constant” amount of

Caveat

Find the pointer to the node before inserting/ removing —> traversal: O(n) - *depends on number of elements in list*

INSERT

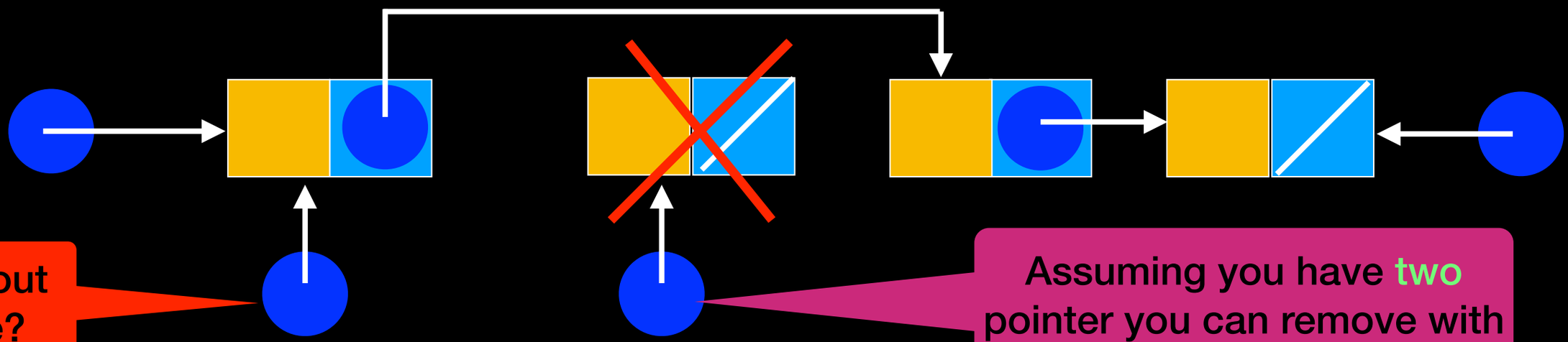
```
void insert(size_t position, T new_element);
```



Assuming you have this pointer you can insert with “constant” amount of work

REMOVE

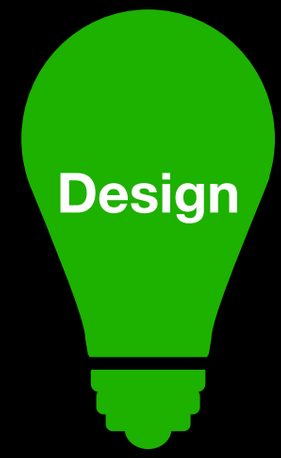
```
void remove(size_t position);
```



What about this one?

Assuming you have two pointer you can remove with “constant” amount of work

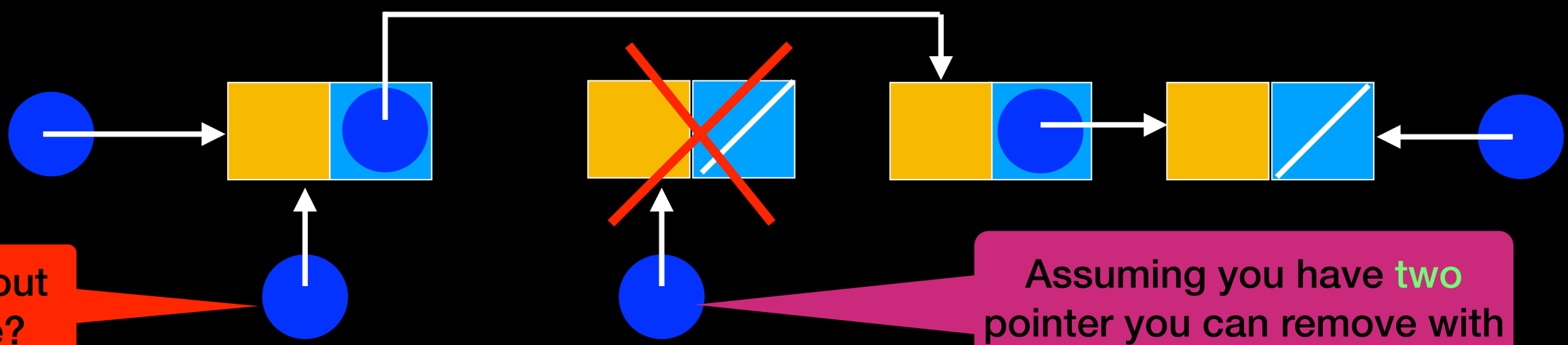
Lecture Activity



Propose a solution to this problem:
In English write a few sentences describing the changes you would make to the Linked-Chain implementation of the List ADT to remove any node in the chain

REMOVE

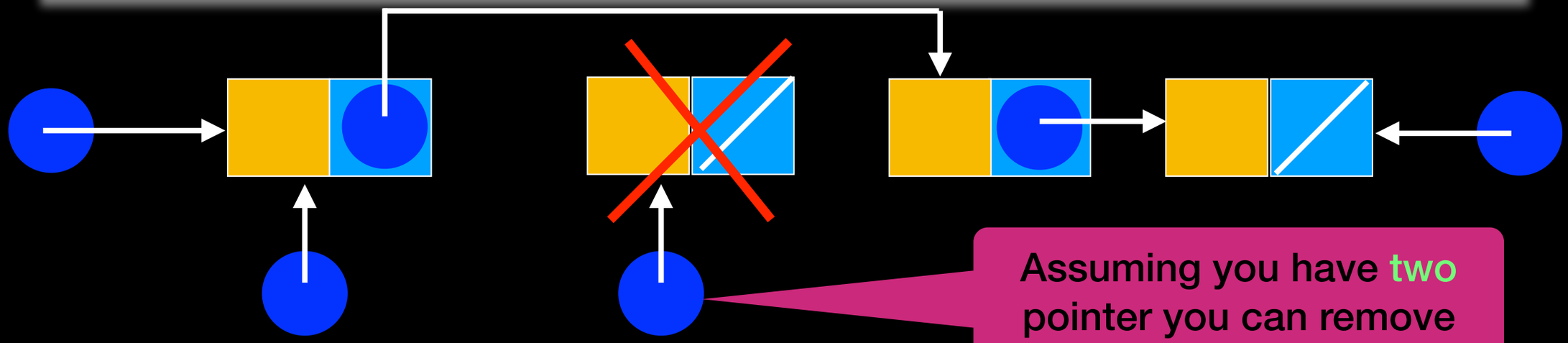
```
void remove(size_t position);
```



One possible solution:
Remove makes **two calls to** `getPointerTo`,
One with `position` and another with `position-1`

REMOVE

```
void remove(size_t position);
Node<T>* getPointerTo(size_t position);
```

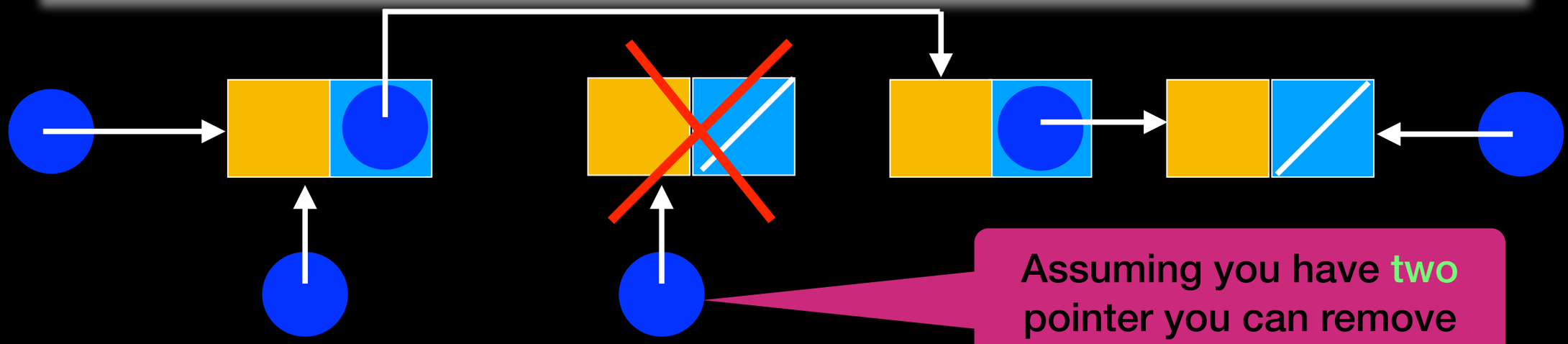


Assuming you have **two** pointer you can remove with “**constant**” amount of

Another possible solution:
Remove sets
`position-1 = position->getNext()`

REMOVE

```
void remove(size_t position);
Node<T>* getPointerTo(size_t position);
```

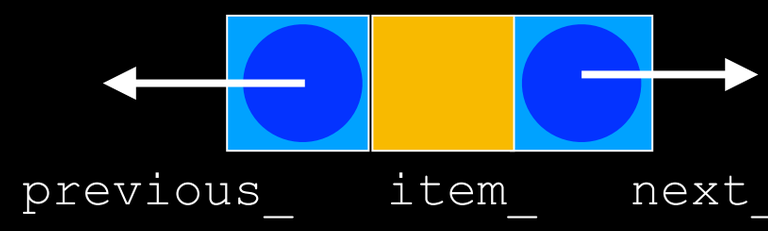


Assuming you have two pointer you can remove with “constant” amount of

Another Approach

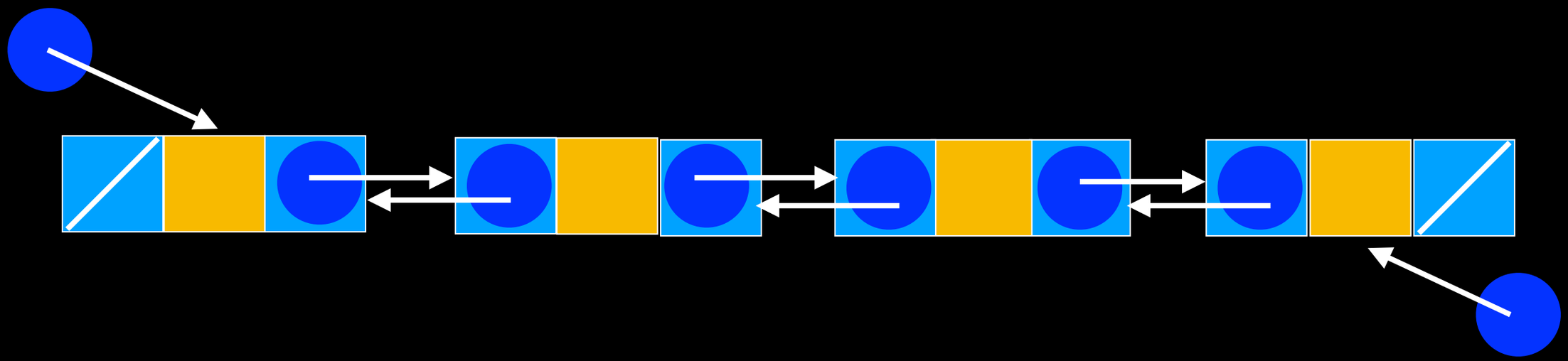
Doubly Linked List

New node with `previous_` and `next_` pointers



```
#ifndef NODE_H_
#define NODE_H_

template<class T>
class Node
{

public:
     Node();
     Node(const T& an_item);
     Node(const T& an_item, Node<T>* next_node_ptr);
     void setItem(const T& an_item);
     void setNext(Node<T>* next_node_ptr);
     void setPrevious(Node<T>* prev_node_ptr);
     T getItem() const;
     Node<T>* getNext() const;
     Node<T>* getPrevious() const;

private:
     T item_;                      // A data item
     Node<T>* next_ ;         // Pointer to next node
     Node<T>* previous_;   // Pointer to previous node
}; // end Node



#include “Node.cpp"
#endif // NODE_H_
```

Doubly Linked List

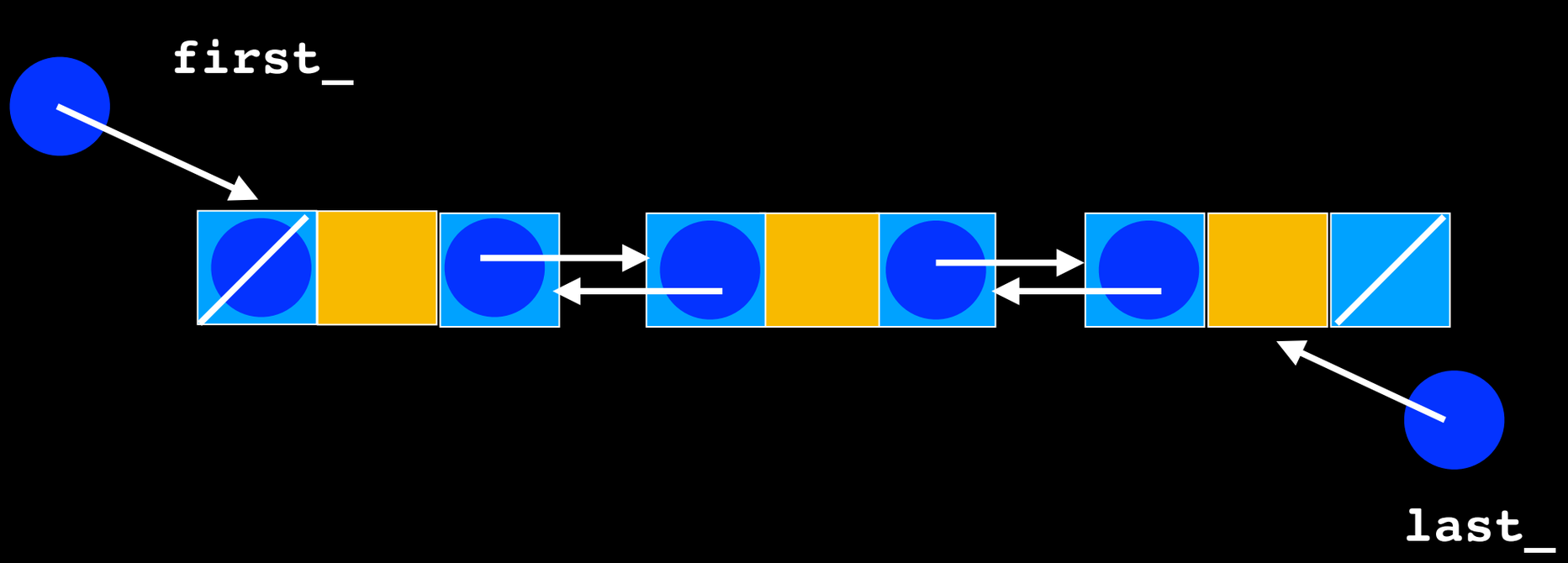


```cpp
#ifndef LIST_H_
#define LIST_H_

#include “Node.hpp"

template<class T>
class List
{

public:
     List(); // constructor
     List(const List<T>& a_list); // copy constructor
     ~List(); // destructor
     bool isEmpty() const;
     size_t getLength() const;
     /retains list order, position is 0 to n-1, if position > n-1 it inserts at end
     bool insert(size_t position, const T& new_element);//retains list order
     bool remove(size_t position);//retains list order
     T getItem(size_t position) const;
     void clear();


private:
     Node<T>* first_;   // Pointer to first node
     Node<T>* last_;    // Pointer to last node
     size_t item_count_;     // number of items in the list

     Node<T>* getPointerTo(size_t position) const;
}; // end List

#include "List.cpp"
#endif // LIST_H_
```

Same interface as Singly-Linked list but uses different node

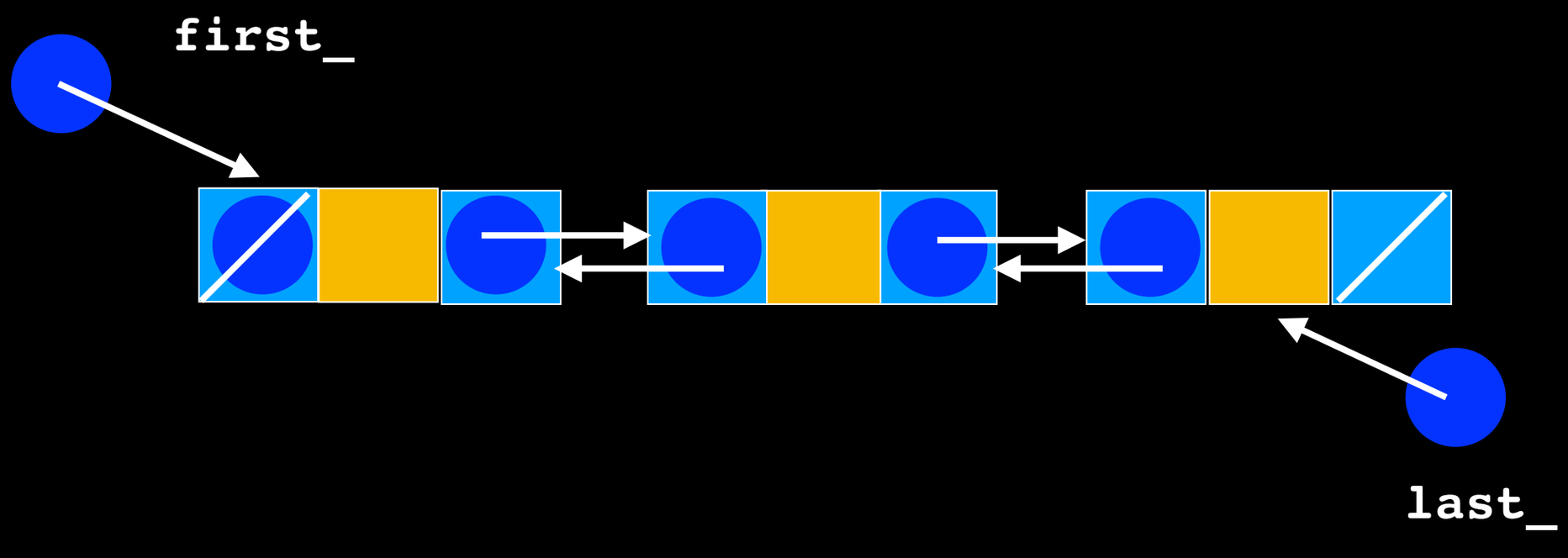
List::insert

What are the different cases that should be considered?

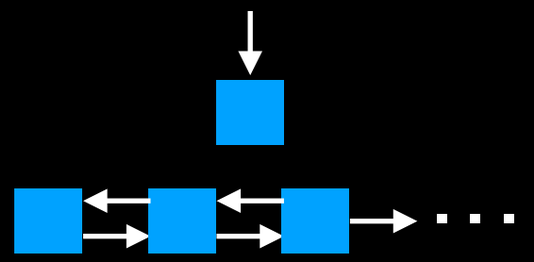


Lecture Activity

Write **Pseudocode** to insert a node at `position > 0 and position < n-1` in a doubly-linked list (assume position follows classic indexing from 0 to item_count - 1)



Pseudocode



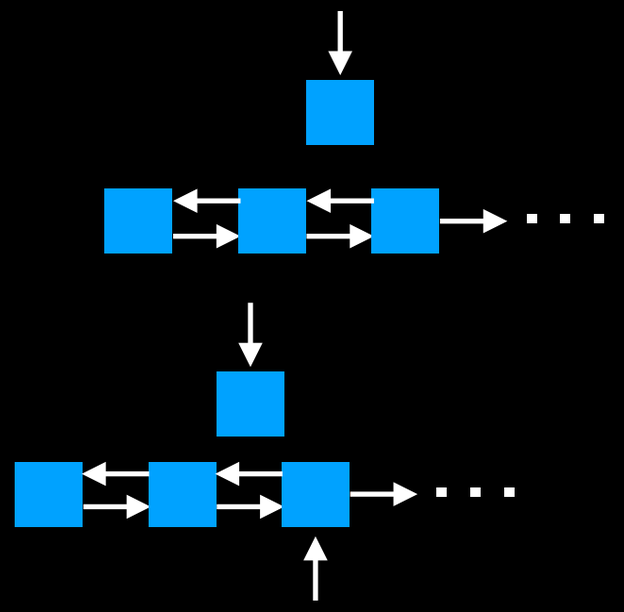
Instantiate new node

Obtain pointer

Connect new node to chain

Reconnect the relevant nodes

Pseudocode



Instantiate new node

Obtain pointer

Connect new node to chain

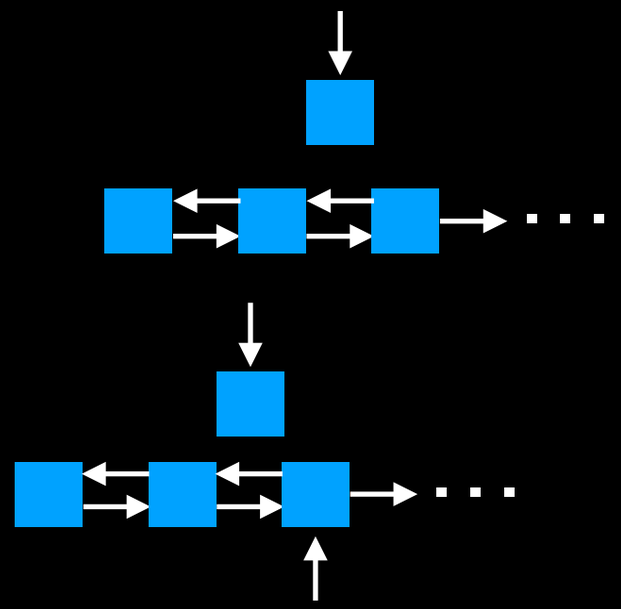
Reconnect the relevant nodes

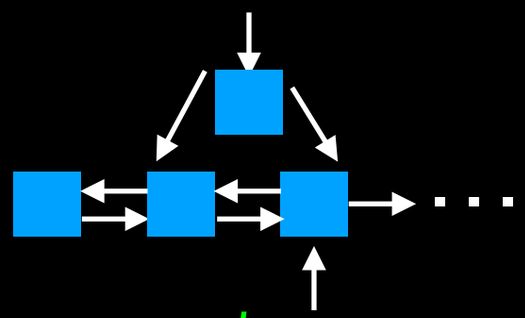
Pseudocode

Instantiate new node

Obtain pointer

Connect new node to chain

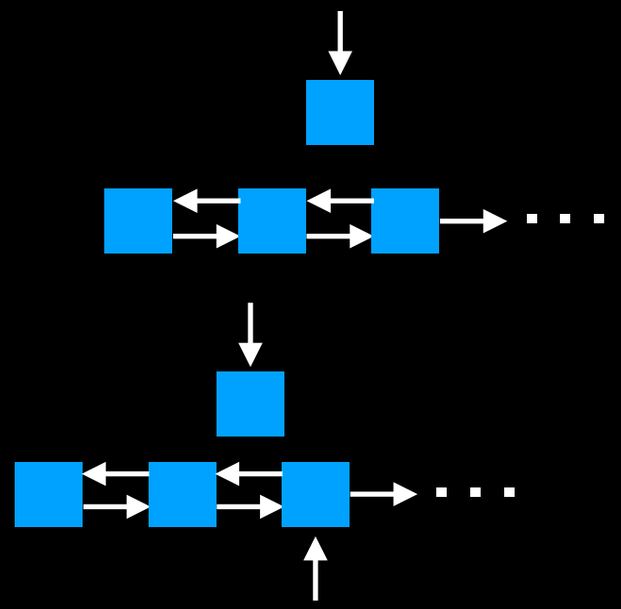




Reconnect the relevant nodes

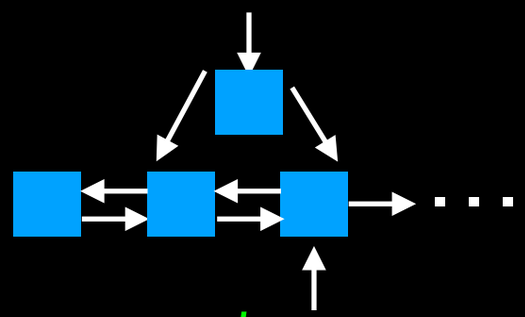
Pseudocode

Instantiate new node

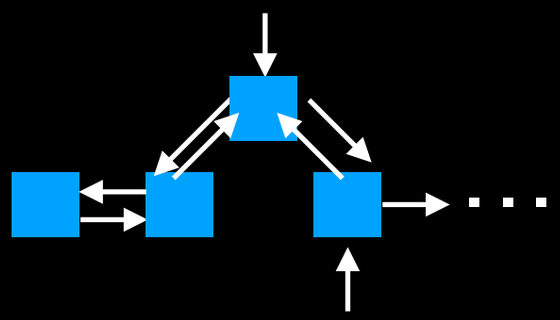


Obtain pointer

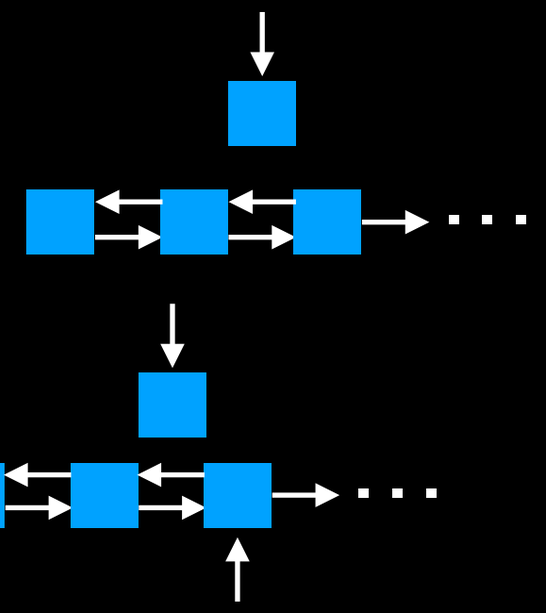
Connect new node to chain



Reconnect the relevant nodes



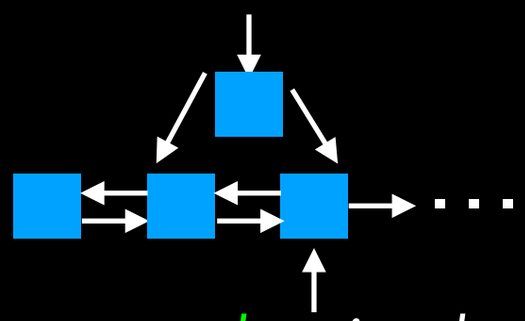
Pseudocode



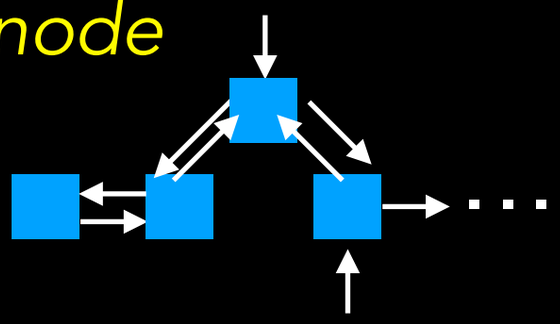
Instantiate new node to be inserted and set its value

Obtain pointer to node currently at position

Connect new node to chain by pointing its next pointer to the node currently at position and its previous pointer to the node at position->previous

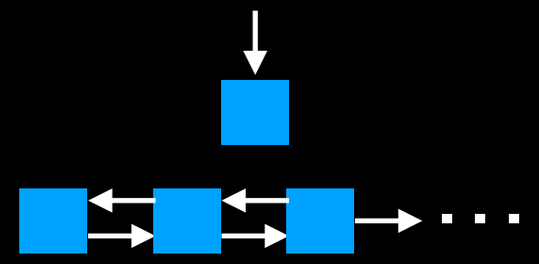


Reconnect the relevant nodes in the chain by pointing position->previous->next to the new node and position->previous to the new node



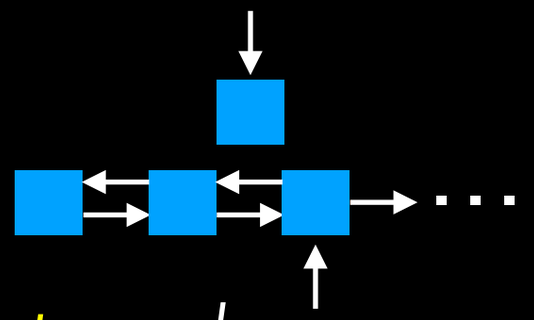
Order Matters!

More Pseudocodey

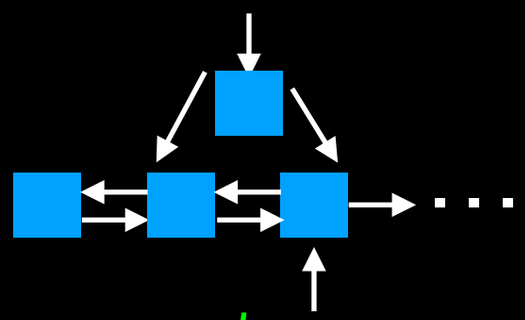


Instantiate new node new_ptr = new Node() and new_ptr->setItem()

Obtain pointer position_ptr = getPointerTo(position)

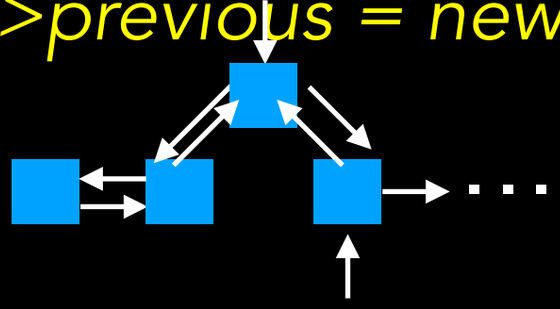


Connect new node to chain new_ptr->next = position_ptr and new_ptr->previous = temp->previous



Reconnect the relevant nodes

position_ptr->previous->next = new_ptr and position->previous = new_ptr



List::insert

```
template<class T>
bool List<T>::insert(size_t position, const T& new_element)
{
      // Create a new node containing the new entry and get a pointer to position
      Node<T>* new_node_ptr = new Node<T>(new_element);
      Node<T>* pos_ptr = getPointerTo(position);

      // Attach new node to chain

   2  else if (pos_ptr == first_)
      {
         // Insert new node at beginning of list
         new_node_ptr->setNext(first_);
         new_node_ptr->setPrevious(nullptr);
         first_->setPrevious(new_node_ptr);
         first_ = new_node_ptr;
      }
   3  else if (pos_ptr == nullptr)
      {
          //insert at end of list
          new_node_ptr->setNext(nullptr);
          new_node_ptr->setPrevious(last_);
          last_->setNext(new_node_ptr);
          last_ = new_node_ptr;
      }
   4  else
      {
         // Insert new node before node to which position points
         new_node_ptr->setNext(pos_ptr);
         new_node_ptr->setPrevious(pos_ptr->getPrevious());
         pos_ptr->getPrevious()->setNext(new_node_ptr);
         pos_ptr->setPrevious(new_node_ptr);
      }  // end if

      item_count_++;  // Increase count of entries
      return true;
}  // end insert
```

O(n)

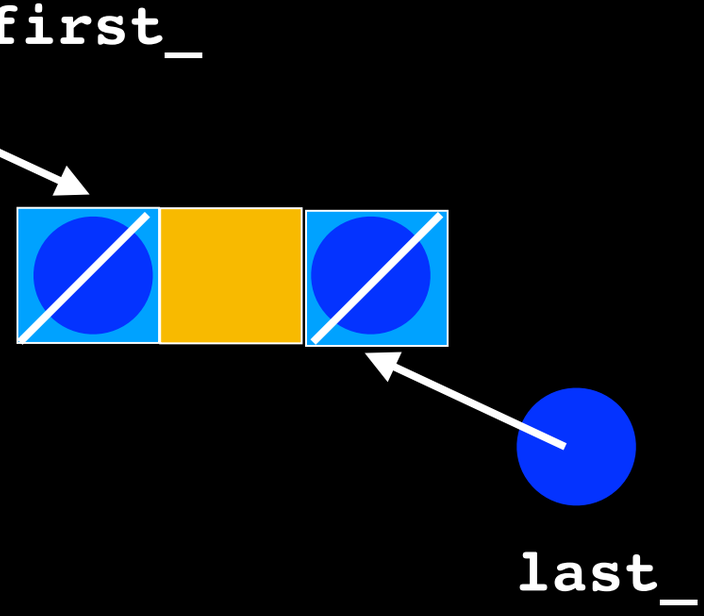
```
if (first_ == nullptr)
 {
      // Insert first node
1     new_node_ptr->setNext(nullptr);
      new_node_ptr->setPrevious(nullptr);
      first_ = new_node_ptr;
      last_ = new_node_ptr;
 }
```

4 O(1) cases

Always insert

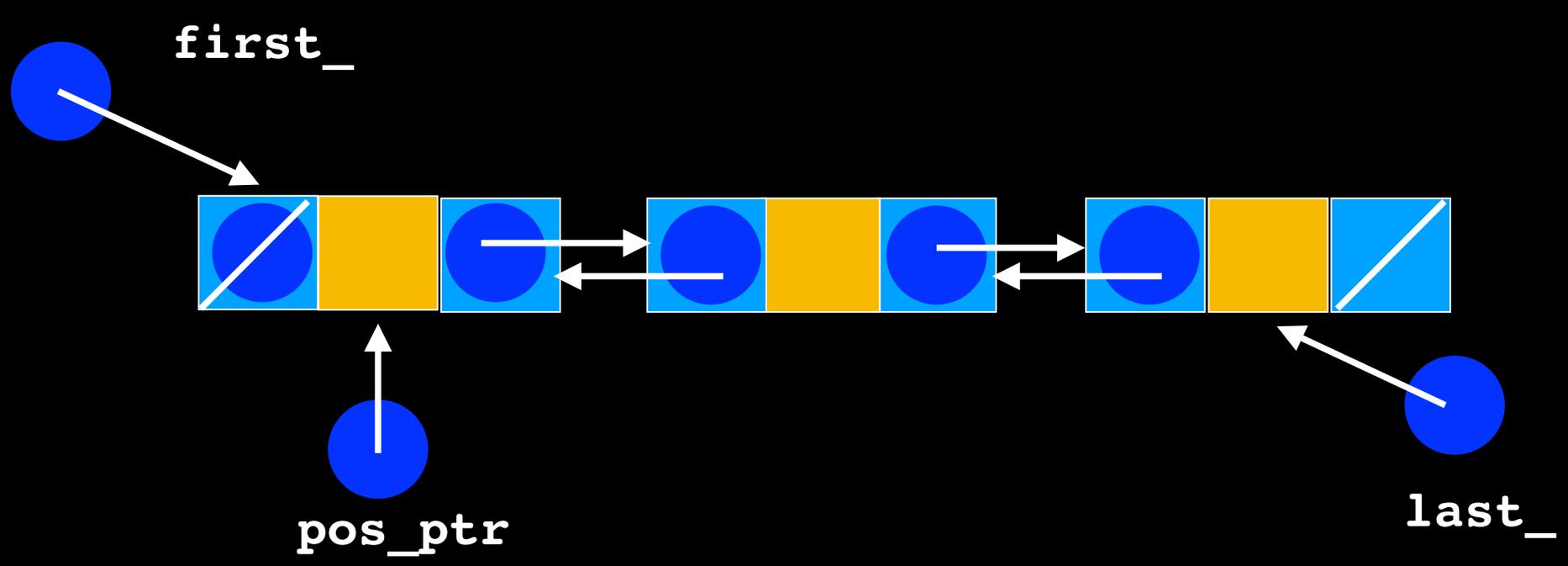
1

```
if (first_ == nullptr)
{
    // Insert first node
    new_node_ptr->setNext(nullptr);
    new_node_ptr->setPrevious(nullptr);
    first_ = new_node_ptr;
    last_ = new_node_ptr;
}
```



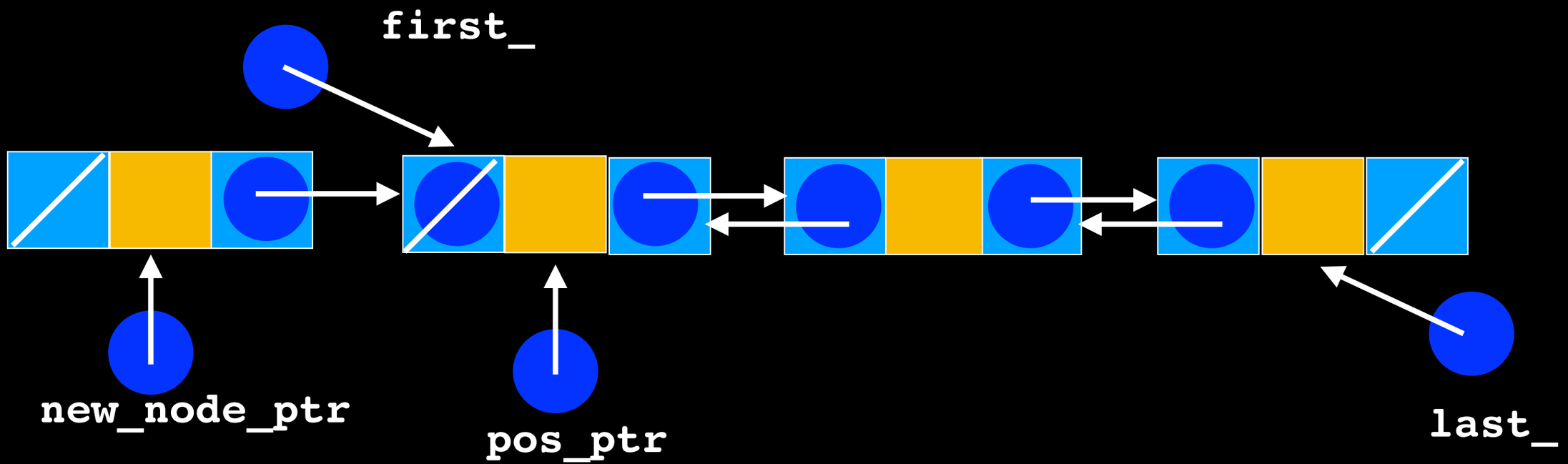
2

```
else if (pos_ptr == first_)
        {
            // Insert new node at beginning of chain
            new_node_ptr->setNext(first_);
            new_node_ptr->setPrevious(nullptr);
            first_->setPrevious(new_node_ptr);
            first_ = new_node_ptr;
        }
```



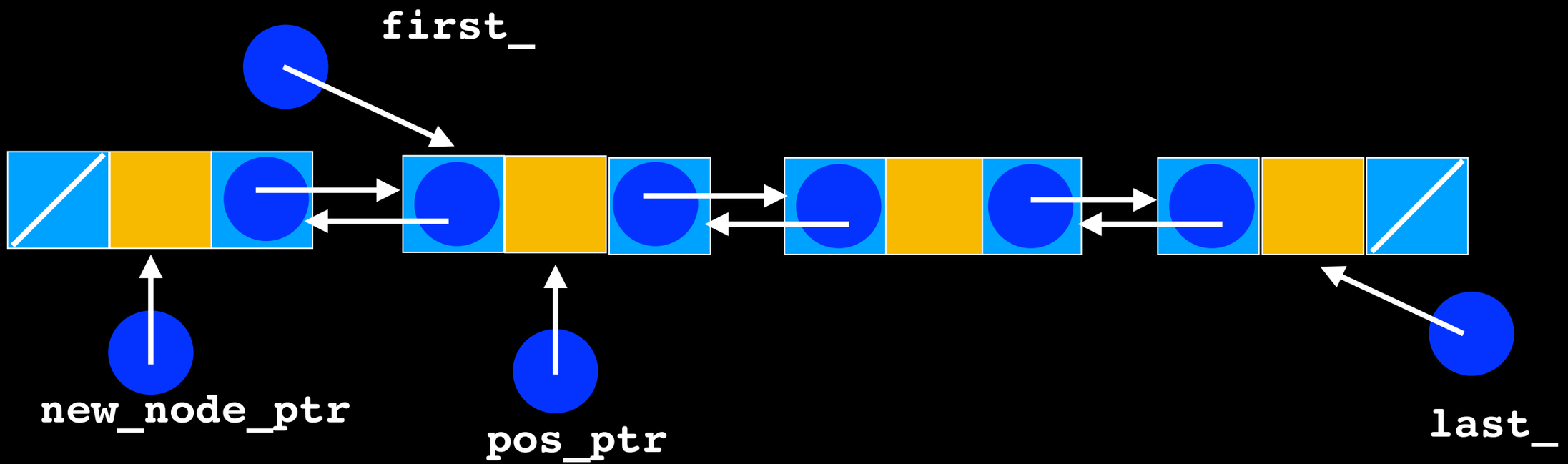
2

```
else if (pos_ptr == first_)
        {
          // Insert new node at beginning of chain
          new_node_ptr->setNext(first_);
          new_node_ptr->setPrevious(nullptr);
          first_->setPrevious(new_node_ptr);
          first_ = new_node_ptr;
        }
```



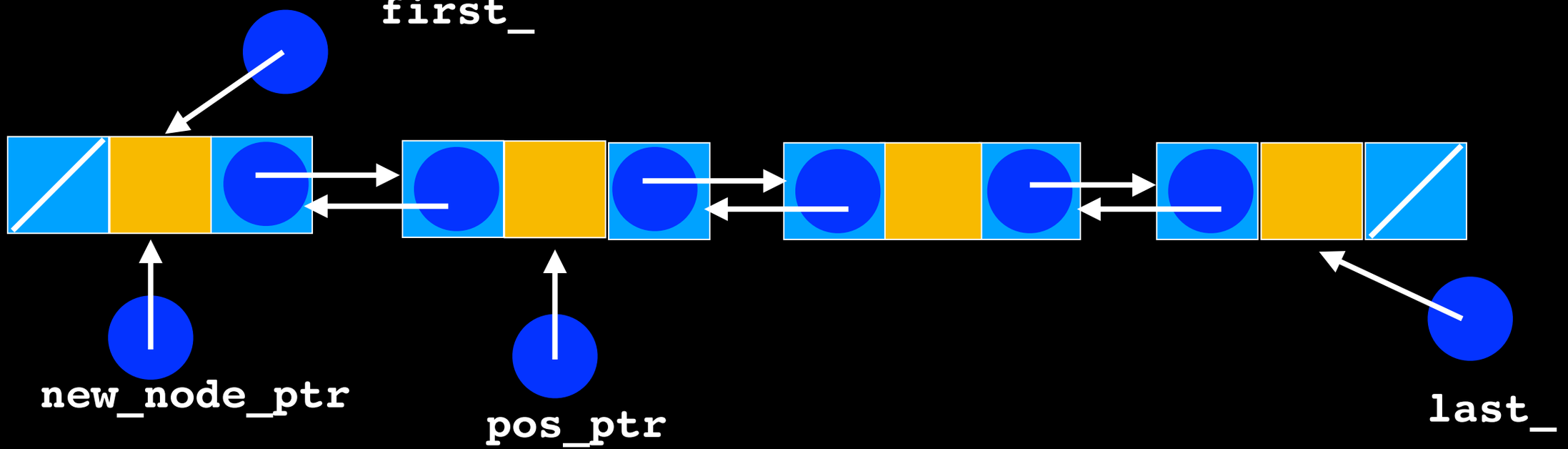
2

```
else if (pos_ptr == first_)
        {
           // Insert new node at beginning of chain
           new_node_ptr->setNext(first_);
           new_node_ptr->setPrevious(nullptr);
           first_->setPrevious(new_node_ptr);
           first_ = new_node_ptr;
        }
```



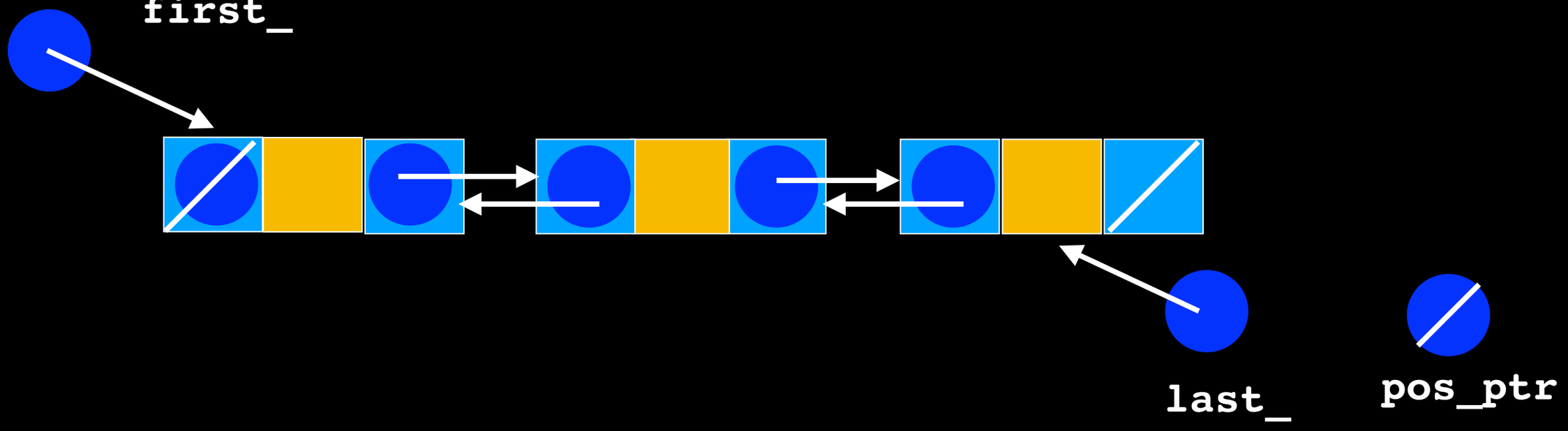
2

```
else if (pos_ptr == first_)
        {
           // Insert new node at beginning of chain
           new_node_ptr->setNext(first_);
           new_node_ptr->setPrevious(nullptr);
           first_->setPrevious(new_node_ptr);
           first_ = new_node_ptr;
        }
```



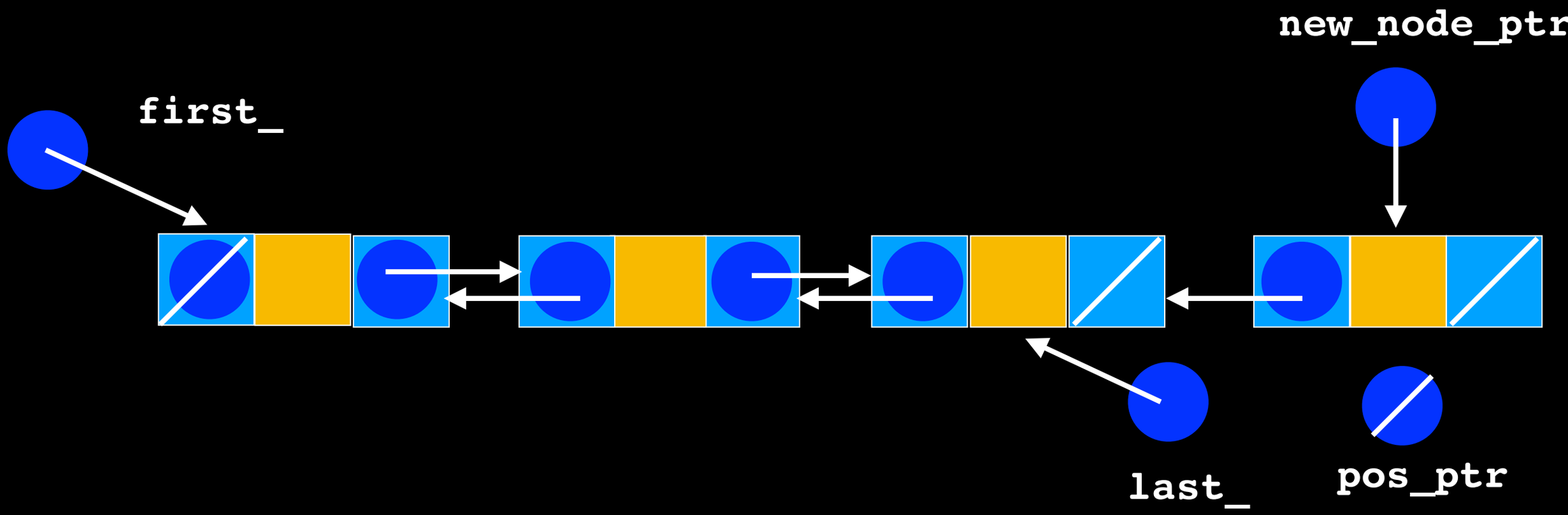
3

```
else if (pos_ptr == nullptr)
{
    //insert at end of list
    new_node_ptr->setNext(nullptr);
    new_node_ptr->setPrevious(last_);
    last_->setNext(new_node_ptr);
    last_ = new_node_ptr;
}
```

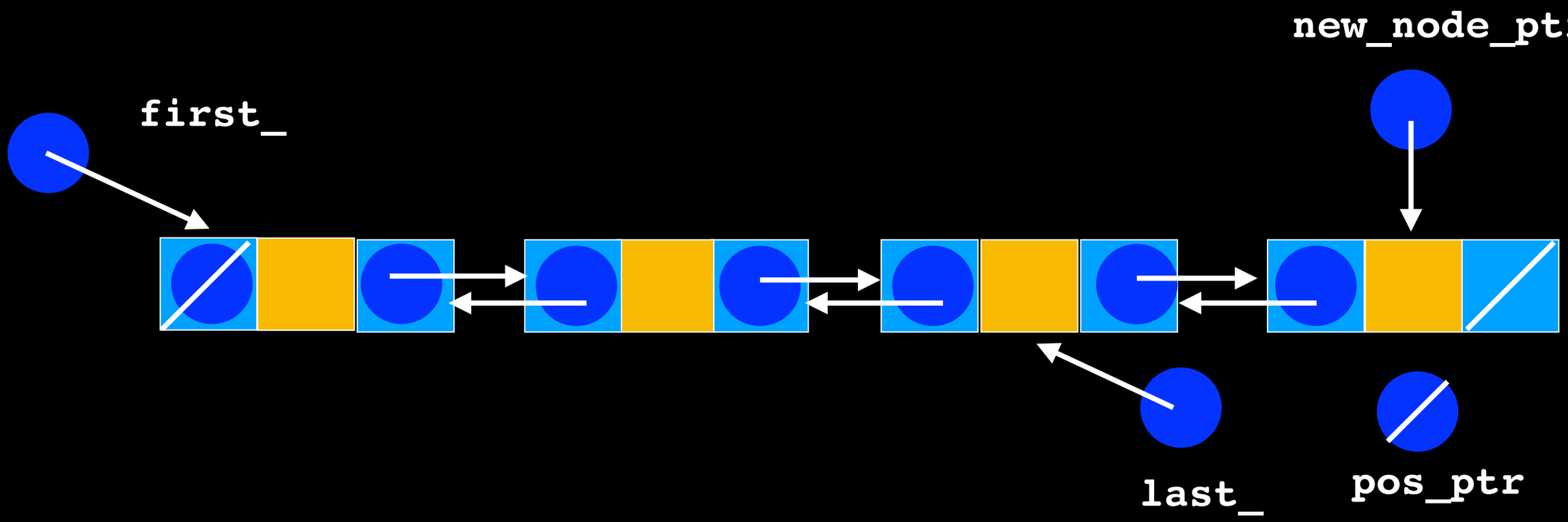


3

```
else if (pos_ptr == nullptr)
{
    //insert at end of list
    new_node_ptr->setNext(nullptr);
    new_node_ptr->setPrevious(last_);
    last_->setNext(new_node_ptr);
    last_ = new_node_ptr;
}
```

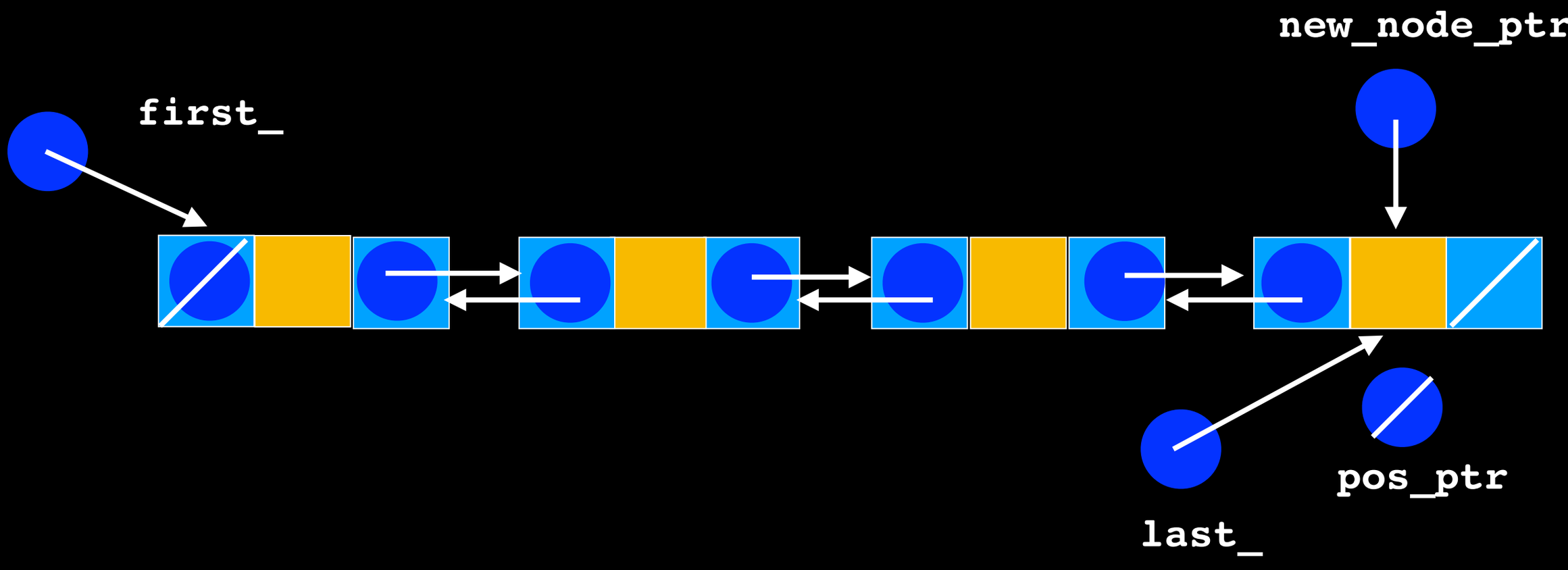


```
3  else if (pos_ptr == nullptr)
   {
       //insert at end of list
       new_node_ptr->setNext(nullptr);
       new_node_ptr->setPrevious(last_);
       last_->setNext(new_node_ptr);
       last_ = new_node_ptr;
   }
```

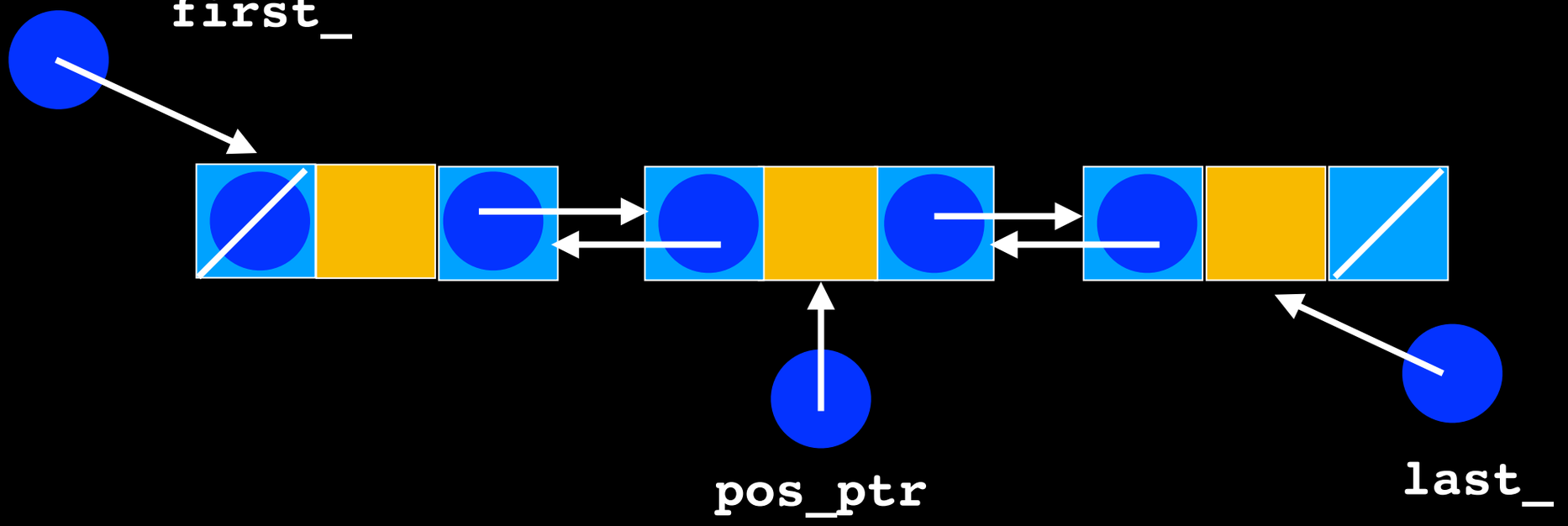


3

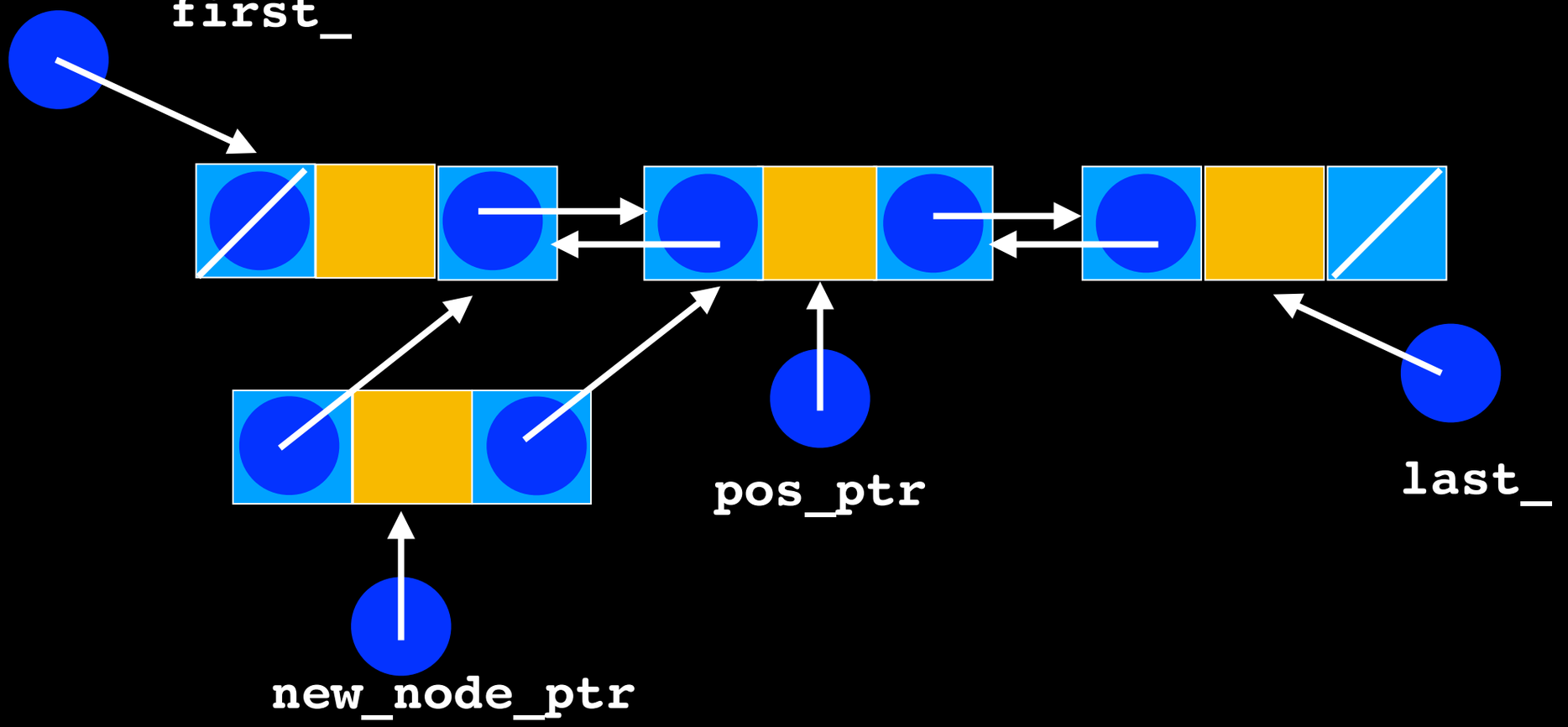
```
else if (pos_ptr == nullptr)
{
    //insert at end of list
    new_node_ptr->setNext(nullptr);
    new_node_ptr->setPrevious(last_);
    last_->setNext(new_node_ptr);
    last_ = new_node_ptr;
}
```



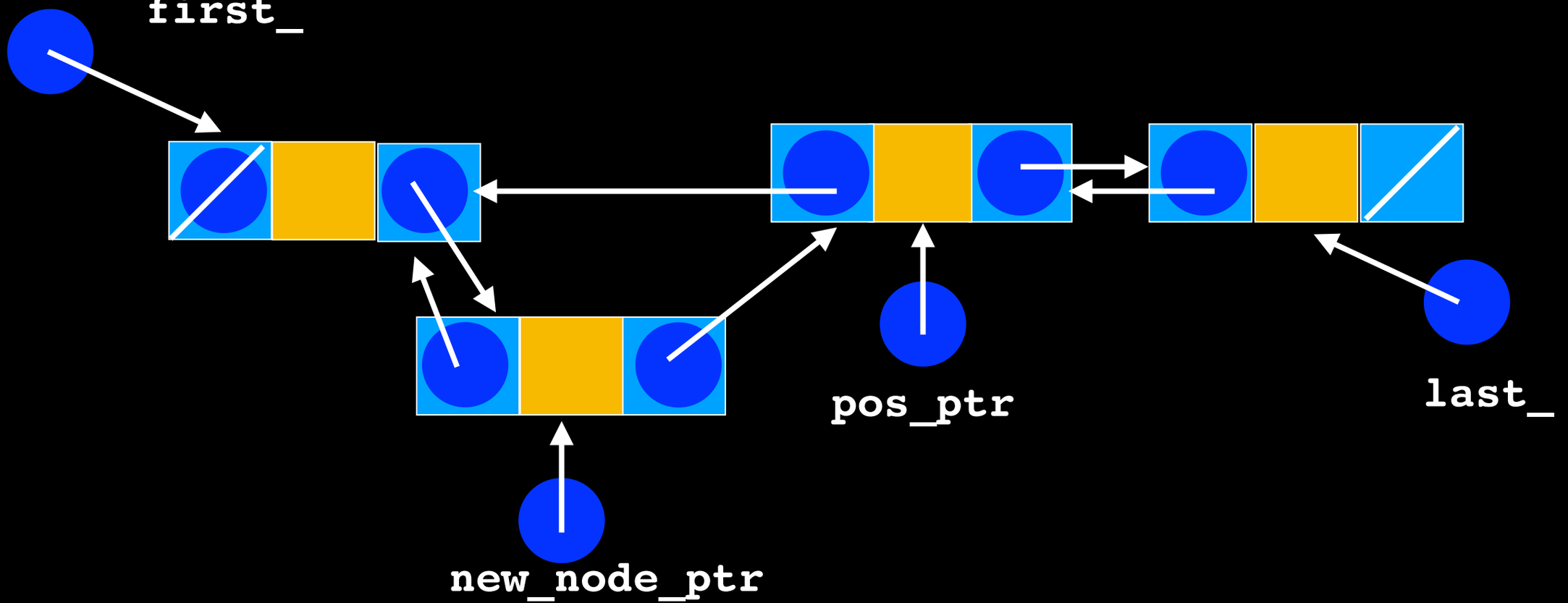
```
4 else
      {
         // Insert new node before node to which position points
         new_node_ptr->setNext(pos_ptr);
         new_node_ptr->setPrevious(pos_ptr->getPrevious());
         pos_ptr->getPrevious()->setNext(new_node_ptr);
         pos_ptr->setPrevious(new_node_ptr);
      }  // end if
```



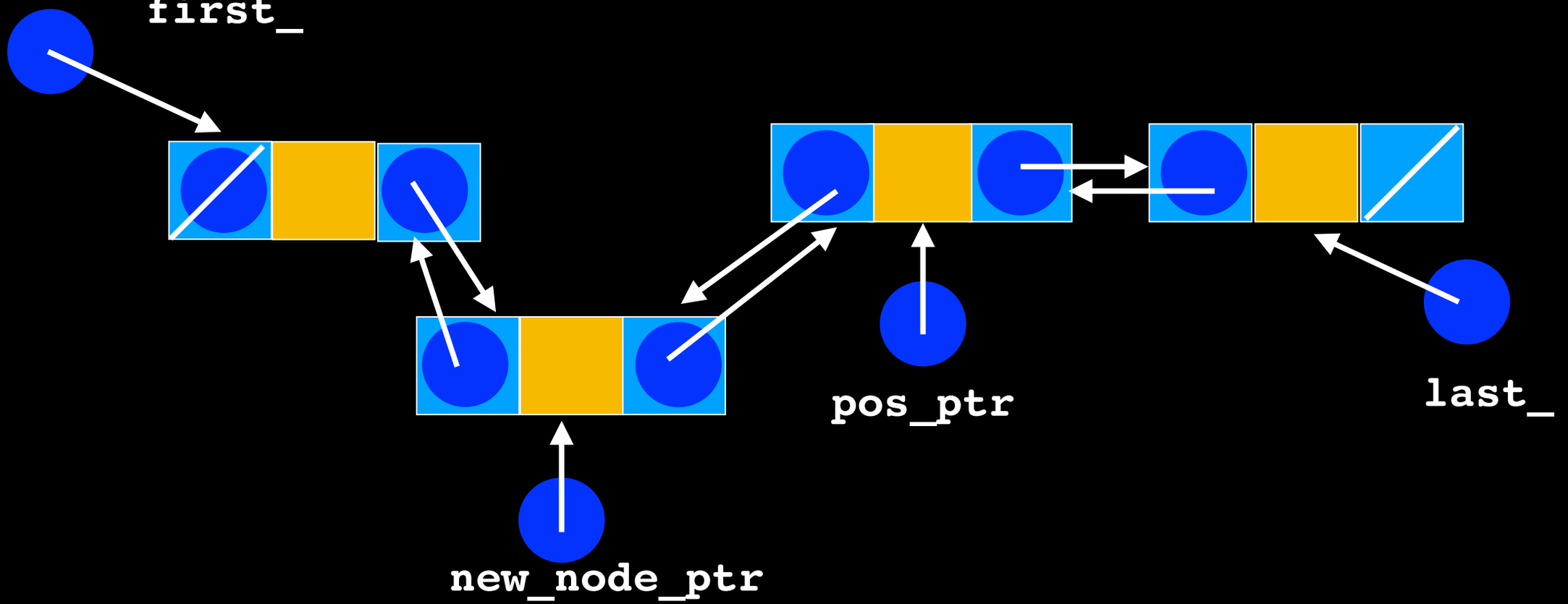
```
4 else
      {
         // Insert new node before node to which position points
         new_node_ptr->setNext(pos_ptr);
         new_node_ptr->setPrevious(pos_ptr->getPrevious());
         pos_ptr->getPrevious()->setNext(new_node_ptr);
         pos_ptr->setPrevious(new_node_ptr);
      }  // end if
```



```
4 else
      {
         // Insert new node before node to which position points
         new_node_ptr->setNext(pos_ptr);
         new_node_ptr->setPrevious(pos_ptr->getPrevious());
         pos_ptr->getPrevious()->setNext(new_node_ptr);
         pos_ptr->setPrevious(new_node_ptr);
      }  // end if
```

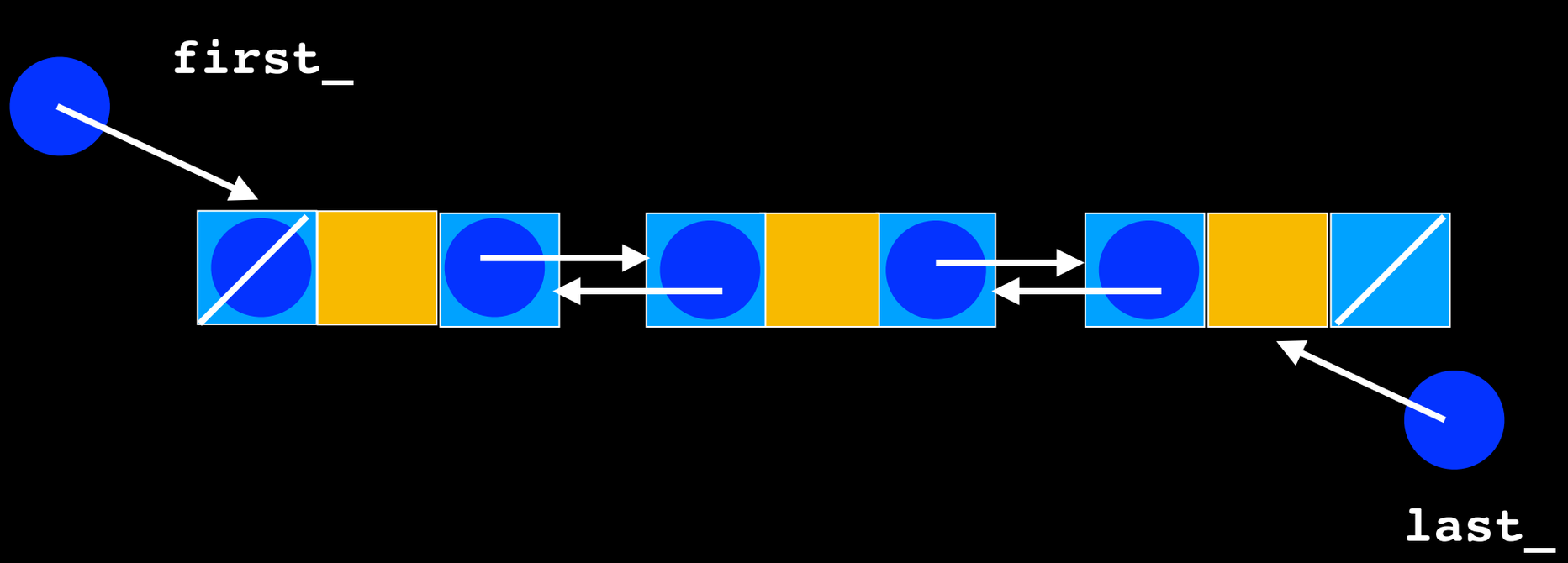


```
4 else
      {
         // Insert new node before node to which position points
         new_node_ptr->setNext(pos_ptr);
         new_node_ptr->setPrevious(pos_ptr->getPrevious());
         pos_ptr->getPrevious()->setNext(new_node_ptr);
         pos_ptr->setPrevious(new_node_ptr);
      }  // end if
```



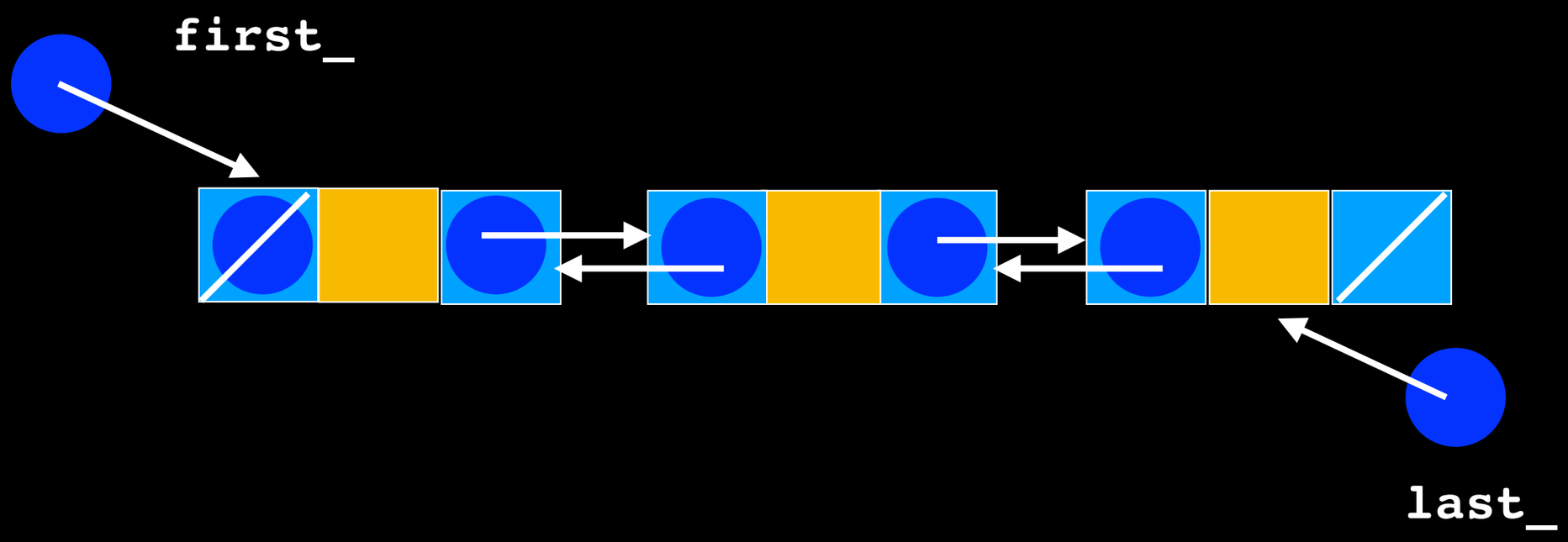
List::remove

What are the different cases that should be considered?



Lecture Activity

Write **Pseudocode** to remove the node at position 1 in a doubly-linked list (assume position follows classic indexing from 0 to item_count - 1, and there is a node at position 2)



List::Remove

```
template<class T>
bool List<T>::remove(size_t position)
{
    // get pointer to position
    Node<T>* pos_ptr = getPointerTo(position);
    if (pos_ptr == nullptr) // no node at position
       return false;
    else
    {
       // Remove node from chain


       else if (pos_ptr == last_ )
       {
          //remove last_  node
          last_  = pos_ptr->getPrevious();
          last_ ->setNext(nullptr);

          // Return node to the system
          pos_ptr->setPrevious(nullptr);
          delete pos_ptr;
          pos_ptr = nullptr;
       }
       else
       {
          //Remove from the middle
          pos_ptr->getPrevious()->setNext(pos_ptr->getNext());
          pos_ptr->getNext()->setPrevious(pos_ptr->getPrevious());

          // Return node to the system
          pos_ptr->setNext(nullptr);
          pos_ptr->setPrevious(nullptr);
          delete pos_ptr;
          pos_ptr = nullptr;
       }
       item_count_--;
       return true;
    }
}  // end remove
```

0 O(n)

1

```
if ((pos_ptr == first_) && pos_ptr == last_))
{
     // Only one node
     first_ = nullptr;
     last_ = nullptr;

     // Return node to the system
     pos_ptr->setNext(nullptr);
     delete pos_ptr;
     pos_ptr = nullptr;
}
```

2

```
if (pos_ptr == first_)
{
     // Remove first node
     first_ = pos_ptr->getNext();
     first_->setPrevious(nullptr);

     // Return node to the system
     pos_ptr->setNext(nullptr);
     delete pos_ptr;
     pos_ptr = nullptr;
}
```

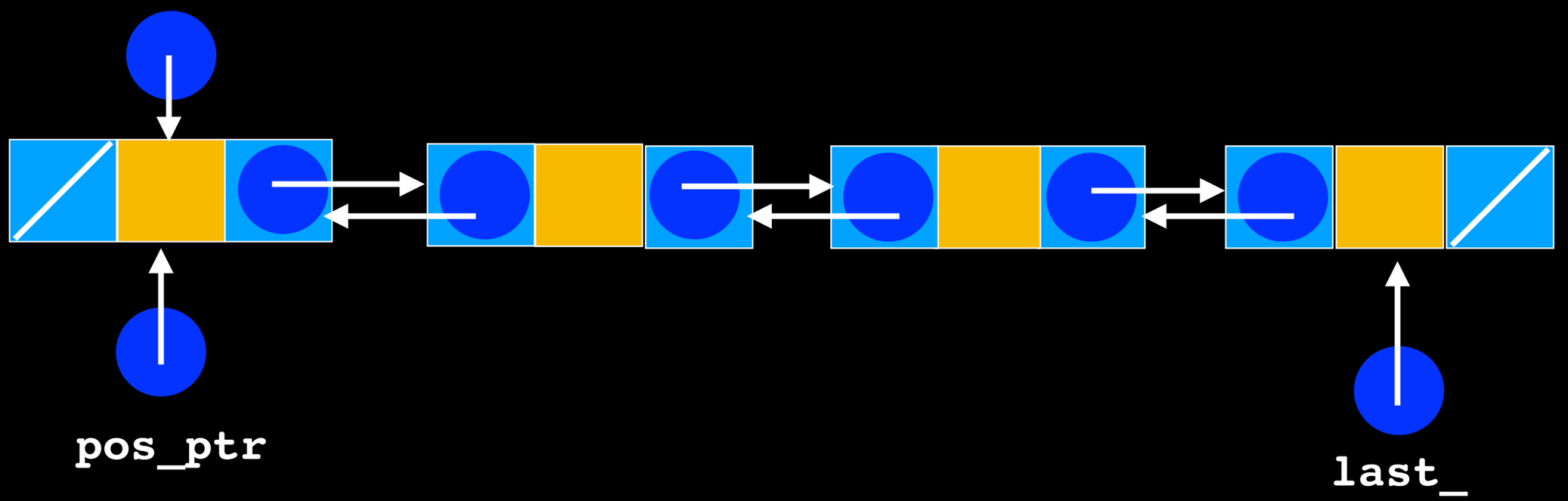
3

4

5 O(1) cases

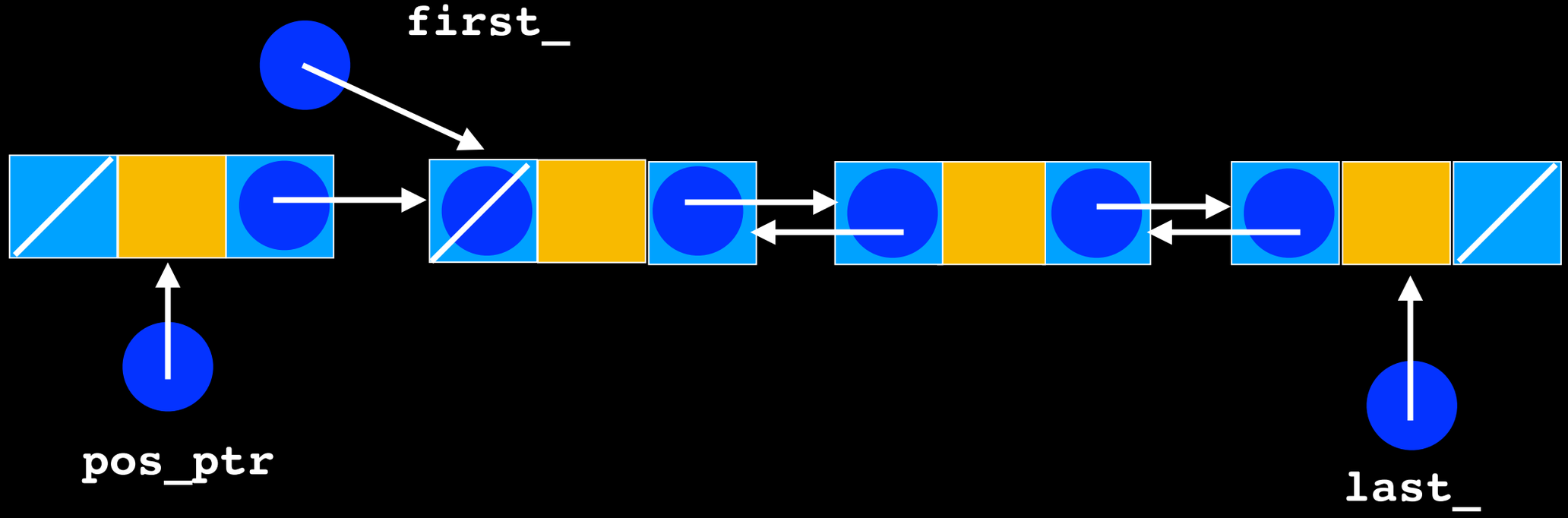
2

```
// Remove node from chain
if (pos_ptr == first_)
{
    // Remove first node
    first_ = pos_ptr->getNext();
    first_->setPrevious(nullptr);

    // Return node to the system
    pos_ptr->setNext(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```



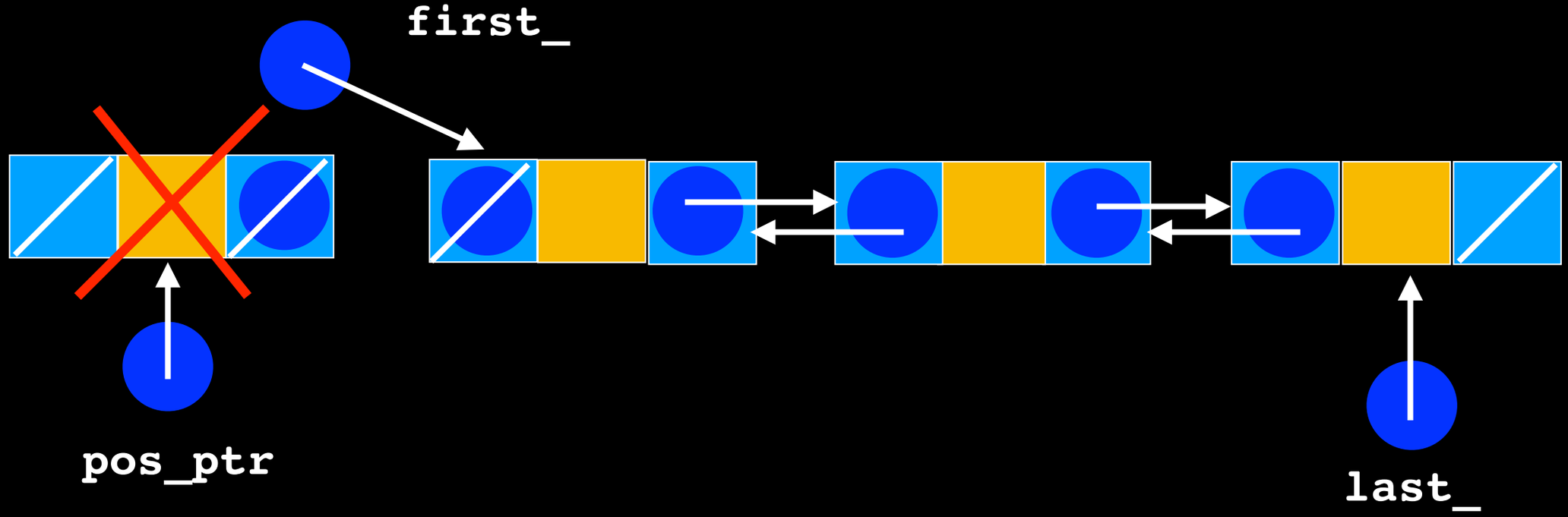
2

```
// Remove node from chain
if (pos_ptr == first_)
{
    // Remove first node
    first_ = pos_ptr->getNext();
    first_->setPrevious(nullptr);

    // Return node to the system
    pos_ptr->setNext(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```



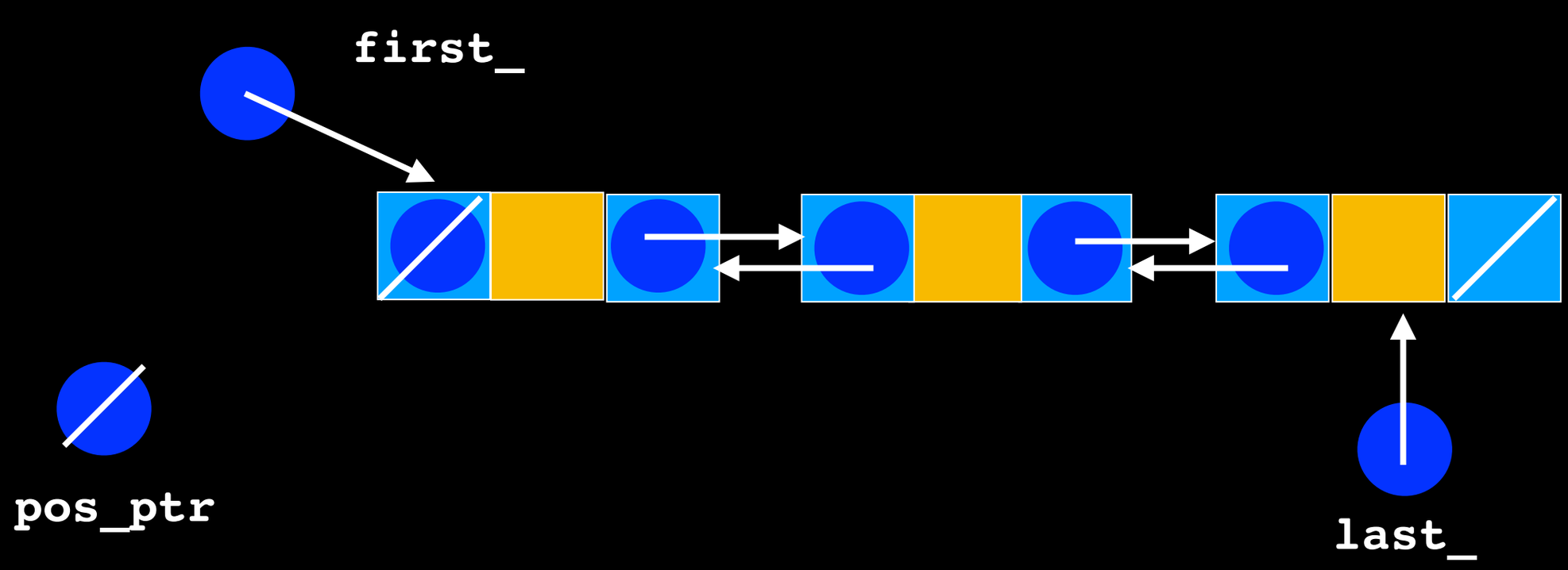
2

```
// Remove node from chain
if (pos_ptr == first_)
{
    // Remove first node
    first_ = pos_ptr->getNext();
    first_->setPrevious(nullptr);

    // Return node to the system
    pos_ptr->setNext(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```

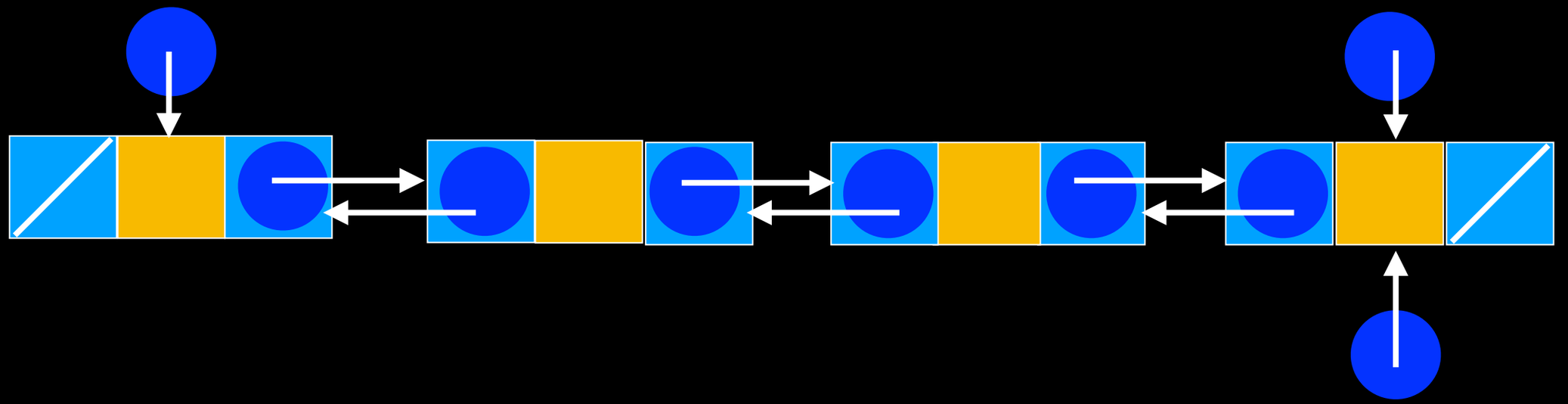


2

```
// Remove node from chain
if (pos_ptr == first_)
{
    // Remove first node
    first_ = pos_ptr->getNext();
    first_->setPrevious(nullptr);

    // Return node to the system
    pos_ptr->setNext(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```

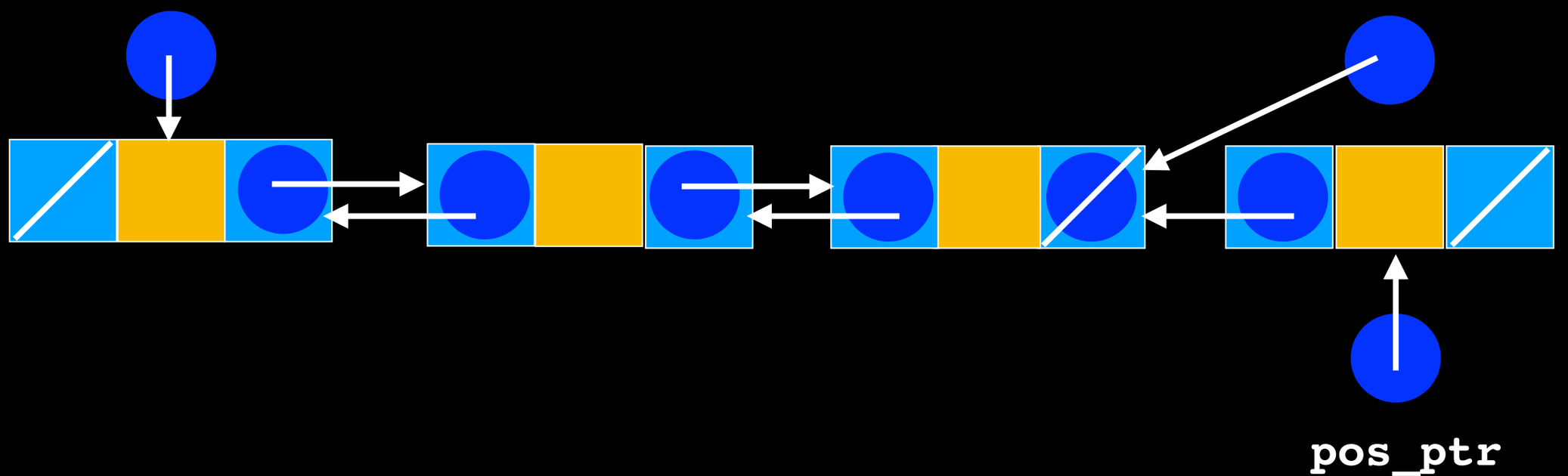


3
```
else if (pos_ptr == last_ )
{
    //remove last_  node
    last_  = pos_ptr->getPrevious();
    last_ ->setNext(nullptr);

    // Return node to the system
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```



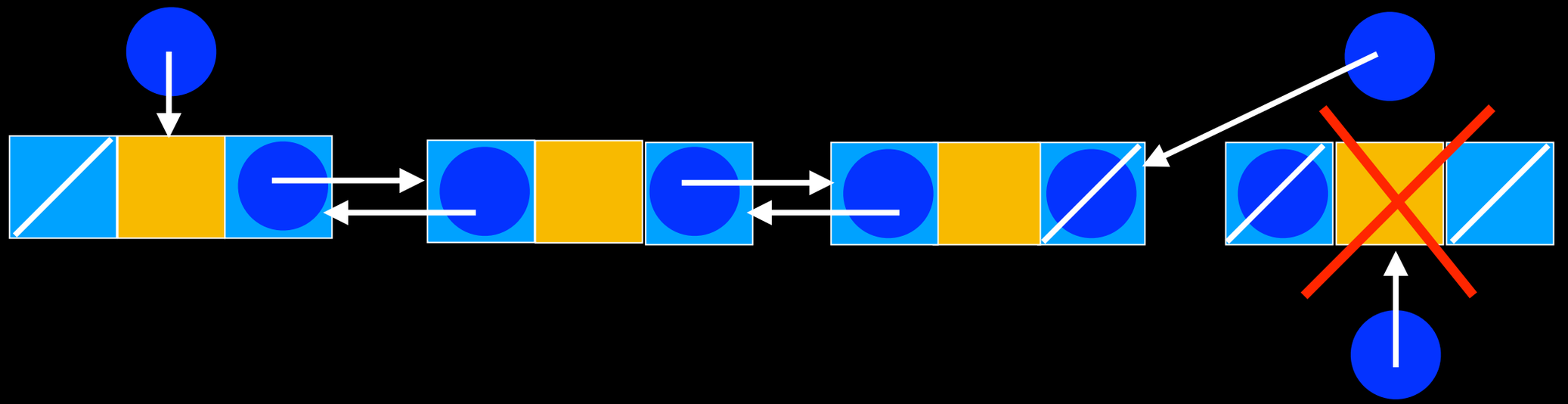
3

```
else if (pos_ptr == last_ )
{
    //remove last_  node
    last_  = pos_ptr->getPrevious();
    last_ ->setNext(nullptr);

    // Return node to the system
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```



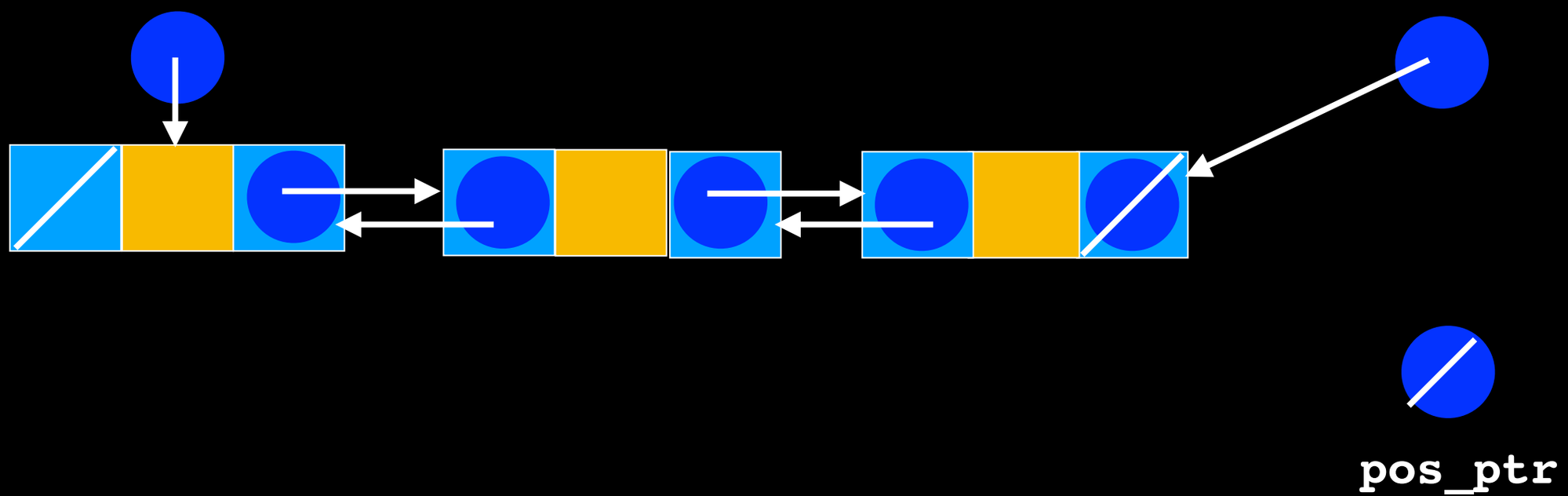
3

```
else if (pos_ptr == last_ )
{
    //remove last_  node
    last_  = pos_ptr->getPrevious();
    last_ ->setNext(nullptr);

    // Return node to the system
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```

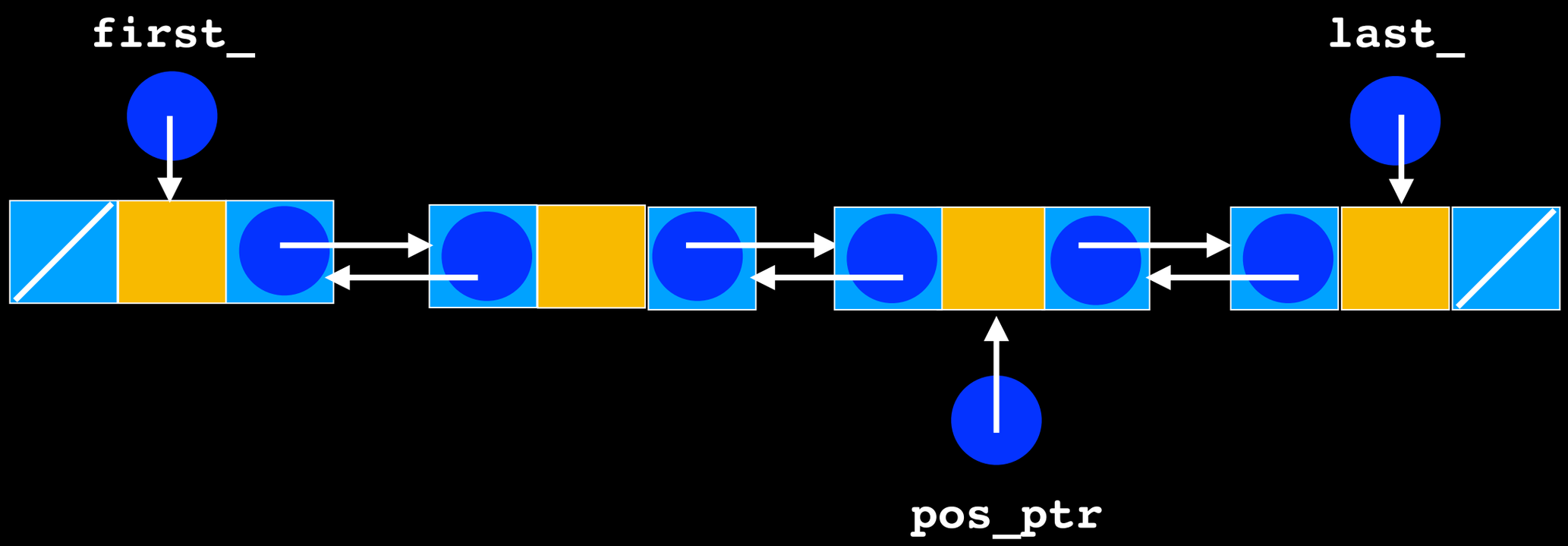


3

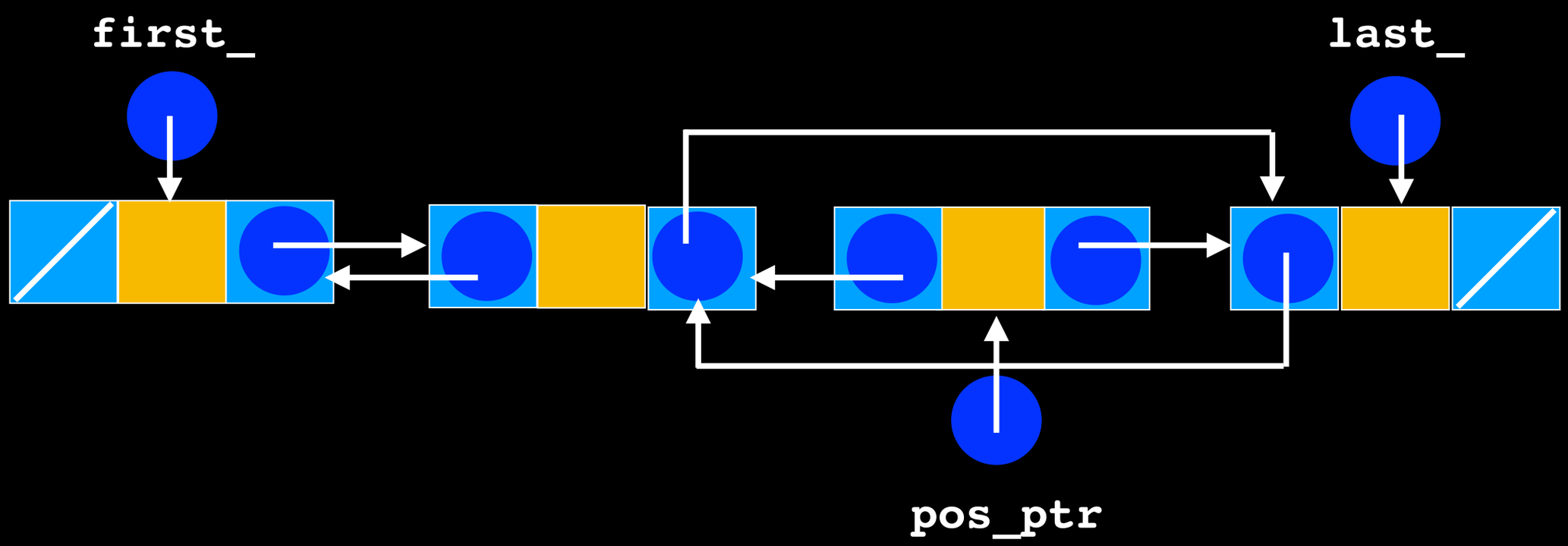
```
else if (pos_ptr == last_ )
{
    //remove last_  node
    last_  = pos_ptr->getPrevious();
    last_ ->setNext(nullptr);

    // Return node to the system
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}
```



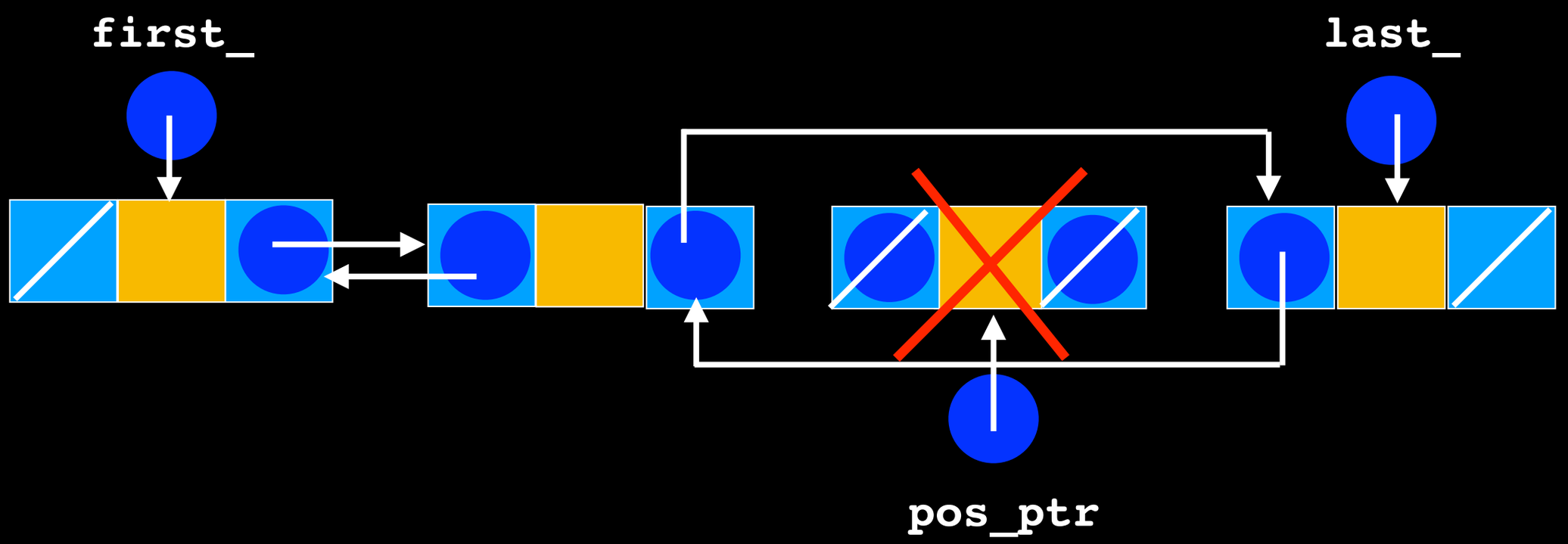
4
```
else if (pos_ptr != nullptr)
{
    //Remove from the middle
    pos_ptr->getPrevious()->setNext(pos_ptr->getNext());
    pos_ptr->getNext()->setPrevious(pos_ptr->getPrevious());

    // Return node to the system
    pos_ptr->setNext(nullptr);
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}  // end if
```



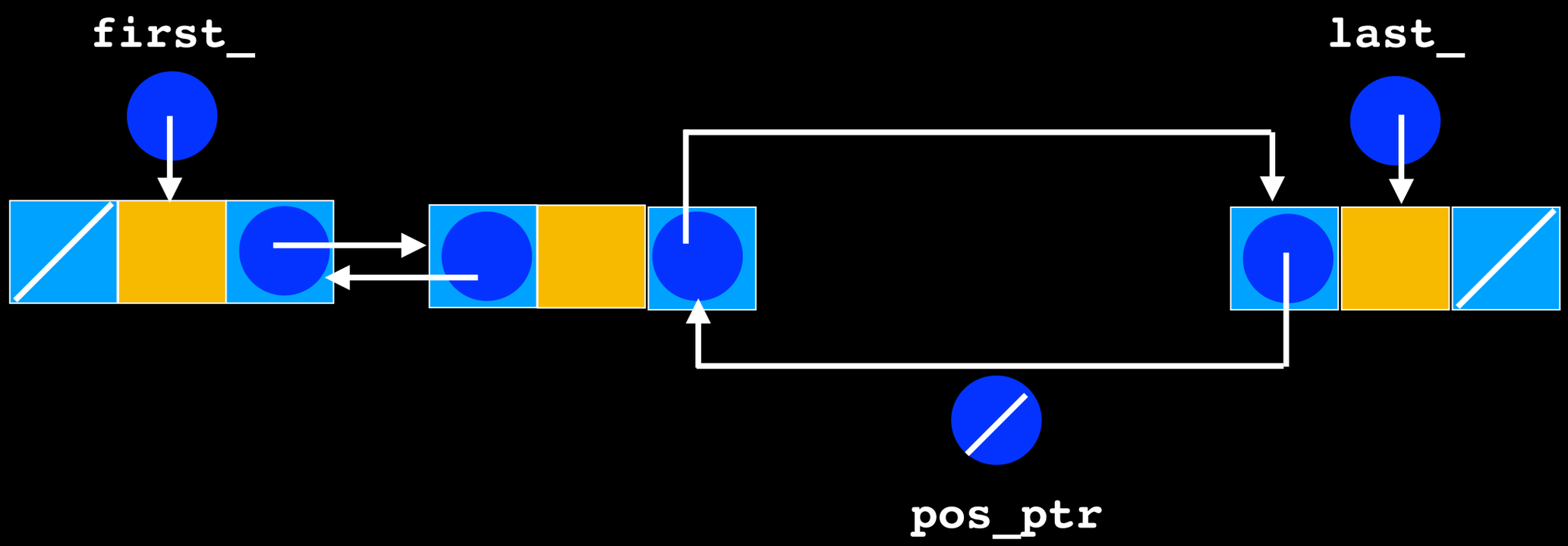
4
```
else if (pos_ptr != nullptr)
{
    //Remove from the middle
    pos_ptr->getPrevious()->setNext(pos_ptr->getNext());
    pos_ptr->getNext()->setPrevious(pos_ptr->getPrevious());

    // Return node to the system
    pos_ptr->setNext(nullptr);
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}  // end if
```



4
```
else if (pos_ptr != nullptr)
{
    //Remove from the middle
    pos_ptr->getPrevious()->setNext(pos_ptr->getNext());
    pos_ptr->getNext()->setPrevious(pos_ptr->getPrevious());

    // Return node to the system
    pos_ptr->setNext(nullptr);
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}  // end if
```



4
```
else if (pos_ptr != nullptr)
{
    //Remove from the middle
    pos_ptr->getPrevious()->setNext(pos_ptr->getNext());
    pos_ptr->getNext()->setPrevious(pos_ptr->getPrevious());

    // Return node to the system
    pos_ptr->setNext(nullptr);
    pos_ptr->setPrevious(nullptr);
    delete pos_ptr;
    pos_ptr = nullptr;
}  // end if
```



List::getPointerTo

```
template<class T>
Node<T>* List<T>::getPointerTo(size_t position) const
{

    Node<T>* find_ptr = nullptr;
    // return nullptr if there is no node at position
    if(position < item_count)
    {//there is a node at position
        find_ptr = first_;
        for(size_t i = 0; i < position; ++i)
        {
            find_ptr = find_ptr->getNext();
        }
         //find_ptr points to the node at position
    }

    return find_ptr;
}//end getPointerTo
```

How could you optimize traversal TO POSITION given this implementation?

```
if(position < item_count/2)
    {
        find_ptr = first_;
         . . .
else
{
        find_ptr = last_;
         . . .
```

Still O(n)!!!

List::getItem

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr != nullptr)
        return pos_ptr->getItem();
    else
        ???
}
```

List::getItem

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr != nullptr)
        return pos_ptr->getItem();
    else
        ???
}
```

Problem: return type is T
There is no “default” or null value to indicate uninitialized object

```
template<class T>
T List<T>::getItem(size_t position) const
{

    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr != nullptr)
        return pos_ptr->getItem();
    else
        //MUST RETURN SOMETHING!!!!
}
```

```
template<class T>
T List<T>::getItem(size_t position) const
{
    T dummy;
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr != nullptr)
        return pos_ptr->getItem();
    else
        return dummy;
}
```

If there is no item at position, can we just return a dummy object?

```cpp
template<class T>
T List<T>::getItem(size_t position) const
{
    T dummy;
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr != nullptr)
        return pos_ptr->getItem();
    else
        return dummy; //problem/warning may return
                      // uninitialized object
}
```

The calling function has no way of knowing the returned object is uninitialized -> UNDEFINED BEHAVIOR

Fail-safe Programming

What happens when preconditions are not met or input data is malformed?

- Do nothing
- Return false - `bool add(const T& newEntry);`
- Use sentinel value: return error codes

Fail-safe Programming

What happens when preconditions are not met or input data is malformed?

- Do nothing

???

- Return false - `bool add(const T& newEntry);`
- Use sentinel value: return error codes

Fail-safe Programming

What happens when preconditions are not met or input data is malformed?

- Do nothing
- Return false - `bool add(const T& newEntry);`

Rely on user to handle problem

- Use sentinel value: return error codes

Rely on user to handle problem

Sometimes it is not possible to return an error code

Fail-safe Programming

What happens when preconditions are not met or input data is malformed?

- Do nothing
- Return false - `bool add(const T& newEntry);`
- Use sentinel value: return error codes

What happens there is no item at position when calling `getItem(size_t position)`?

assert

```
#include <cassert>

// ...
assert(getPointerTo(position) != nullptr);
```

Make sure this is true

If assertion is false, program execution terminates

assert

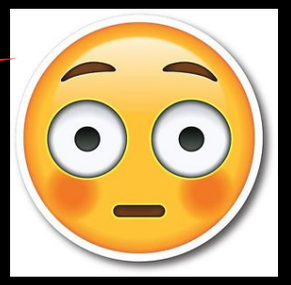
```
#include <cassert>

// ...
assert(getPointerTo(position) != nullptr);
```

Make sure this is true

If assertion is false, program execution terminates

Good for testing and debugging

So drastic! Give me another chance!



Exceptions:

A Light Introduction

Exceptions

Software: calling function

Client might be able to recover from a violation or unexpected condition

Communicate Exception (error) to client:

- Bypass normal execution
- Return control to client
- Communicate error

Exceptions

Client might be able to recover from a violation or unexpected condition

Communicate Exception (error) to client:

- Bypass normal execution
- Return control to client
- Communicate error

Throw and Exception

Throwing Exceptions

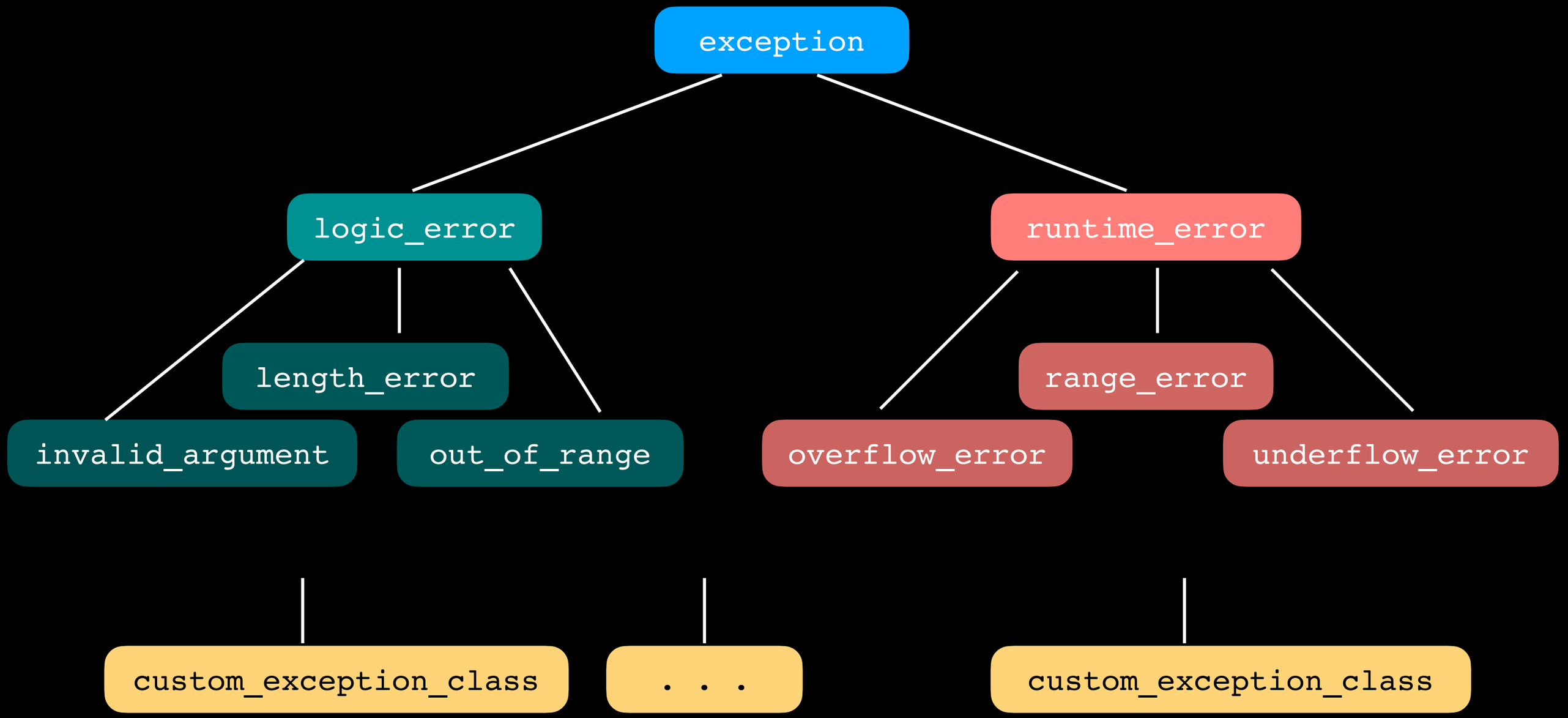
Type of Exception

```
throw( ExceptionClass( stringArgument) )
```

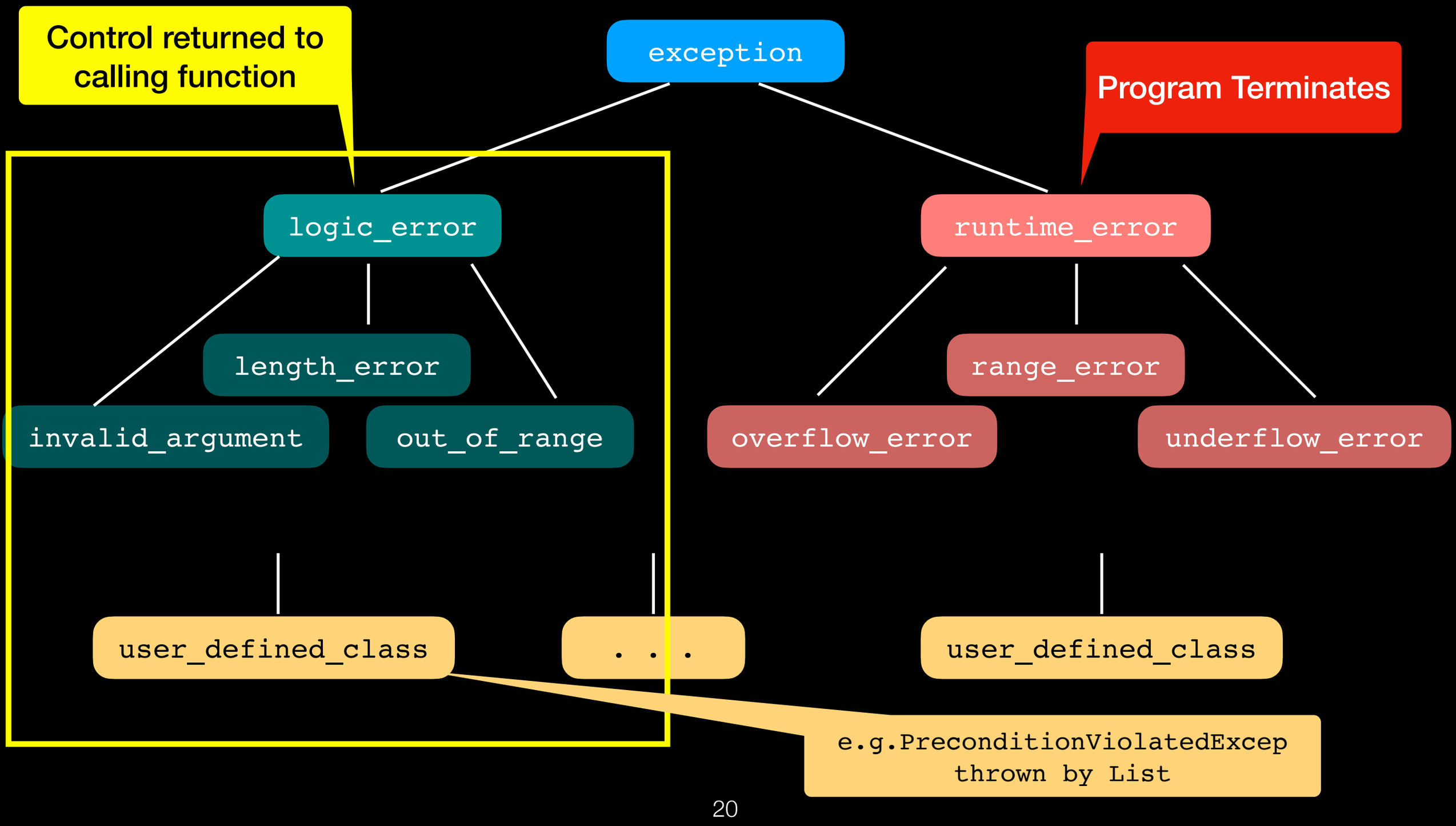
Message describing Exception

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr == nullptr)
            throw (std::out_of_range("getItem called with empty list
                                      or invalid position"));
    else
        return pos_ptr->getItem();
}
```

C++ Exception Classes



C++ Exception Classes



Exception Type		Header File
exception		<exception>
	bad_alloc	<new>
	bad_cast	<typeinfo>
	bad_exception	<exception>
	bad_typeid	<typeinfo>
	ios_base::failure	<ios>
	logic_error	<stdexcept>
	length_error	<stdexcept>
	domain_error	<stdexcept>
	out_of_range	<stdexcept>
	invalid_argument	<stdexcept>
	runtime_error	<stdexcept>
	overflow_error	<stdexcept>
	range_error	<stdexcept>
	underflow_error	<stdexcept>

Exception Handling

Can handle only exceptions of class `logic_error` and its derived classes

Exception Handling Syntax

```
try
{
    //statement(s) that might throw exception
}
catch(ExceptionClass1 identifier)
{
    //statement(s) that react to an exception
   // of type ExceptionClass1
}
```

Exception Handling Syntax

```
try
{
    //statement(s) that might throw exception
}
catch(ExceptionClass1 identifier)
{
    //statement(s) that react to an exception
   // of type ExceptionClass1
}
catch(ExceptionClass2 identifier)
{
    //statement(s) that react to an exception
   // of type ExceptionClass2
}
. . .
```

Exception Handling Syntax

Arrange catch blocks in order of specificity, catching most specific first (i.e. lower in the Exception Class Hierarchy first)

```
try
{
    //statement(s) that might throw exception
}
catch(const ExceptionClass1& identifier)
{
    //statement(s) that react to an exception
    // of type ExceptionClass1
}
catch(const ExceptionClass2& identifier)
{
    //statement(s) that react to an exception
    // of type ExceptionClass2
}
. . .
```

Good practice to catch exceptions by `const reference` whenever possible (due to memory management, avoiding copying and slicing issues)

Exception Handling Usage

You know `getItem()` may throw an exception so call it in a `try` block

```
try
{
     some_object = my_list.getItem(n);
}
catch(const std::out_of_range& problem)
{
     //do something else instead
     bool object_not_found = true;
}
```

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr == nullptr)
        throw(std::out_of_range("getItem called with empty list or invalid position"));
    else
        return pos_ptr->getItem();

}
```

```
try
{
    some_object = my_list.getItem(n);
}
catch(const std::out_of_range& problem)
{
    std::cerr << problem.what() << std::endl;
    //do something else instead
     bool object_not_found = true;
}
```

Returns string parameter to thrown exception

Error Output Stream:
getItem called with empty list or invalid position

Uncaught Exceptions

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr == nullptr)
        throw(std::out_of_range("getItem called with empty list or invalid position"));
    else
        return pos_ptr->getItem();

}
```

out_of_range exception thrown here

```
T someFunction(const List<T>& some_list)
{

    T an_item;
    //code here
    an_item = some_list.getItem(n);

}
```

out_of_range exception not handled here

```
int main()
{
    List<string> my_list;
    try
    {
        std::string some_string = someFunction(my_list);
    }
    catch(const std::out_of_range& problem)
    {
        //code to handle exception here
    }
    //more code here
    return 0;
}
```

out_of_range exception handled here

Uncaught Exceptions

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr == nullptr)
        throw(std::out_of_range("getItem called with empty list or invalid position"));
    else
        return pos_ptr->getItem();

}
```

out_of_range exception thrown here

```
T someFunction(const List<T>& some_list)
{

    T an_item;
    //code here
    an_item = some_list.getItem(n);

}
```

out_of_range exception not handled here

```
int main()
{
    List<string> my_list;
    std::string some_string = someFunction(my_list);
    //code here
    return 0;
}
```

out_of_range exception not handled here

Abnormal program termination

Implications

There could be several
. . . out of the scope of this course

We will discuss one:

What happens when program that dynamically allocated memory relinquishes control in the middle of execution because of an exception?

Implications and Complications

There could be many
. . . out of the scope of this course

We will discuss one:

What happens when program that dynamically allocated memory relinquishes control in the mic' '' of execution because of an exception?

Dynamically allocated memory never released!!!

Implications and Complications

Whenever using dynamic memory allocation and exception handling together must consider ways to prevent memory leaks

Memory Leak

out_of_range exception thrown here

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr == nullptr)
        throw(std::out_of_range("getItem called with empty list or invalid position"));
    else
        return pos_ptr->getItem();

}
```

```
T someFunction(const List<T>& some_list)
{
    //code here that dynamically allocates memory
    T an_item;
    //code here
    an_item = some_list.getItem(n);

}
```

out_of_range exception not handled here

```
int main()
{
    List<string> my_list;
    try
    {
        std::string some_string = someFunction(my_list);
    }
    catch(const std::out_of_range& problem)
    {
        //code to handle exception here
    }
    //more code here
    return 0;
}
```

out_of_range exception handled here